

Robot-Aware Path Planning Benchmark

From Algorithm Validation to Safety-Aware Mission-Level Planner Evaluation

Sooyong Kim
Robotics Engineering Portfolio Project

October 2026

Abstract

This study evaluates path planning as a robot-dependent execution problem rather than as an isolated shortest-path problem. The objective is not to propose a new planning algorithm. Instead, established methods—Dijkstra, A*, RRT, and RRT*—are placed inside a common indoor-navigation benchmark that progressively introduces physical robot footprint, metric configuration space, temporary environmental clutter, path refinement, continuous-clearance validation, robot-specific motion constraints, and a safety-aware operating-speed policy.

The benchmark uses a synthetic metric office/laboratory/warehouse-like environment at 0.05 m/cell and two physically different mobile platforms: TurtleBot3 Waffle Pi and Clearpath Ridgeback. Scenario 1 isolates footprint-driven route divergence. Scenario 2 introduces 132 temporary/caution objects while preserving the same mission. Scenario 3 adds a common 0.025 m safety allowance and compares all four planners through the same path-to-execution pipeline.

Scenario 1 produced final smoothed paths of 42.29 m for Waffle and 48.57 m for Ridgeback, while rectangular-footprint validation confirmed that the Waffle shortcut was physically unavailable to Ridgeback at 285 sampled path locations for every tested 5-degree orientation. Scenario 2 left the Waffle route nearly unchanged at 42.34 m but increased Ridgeback to 79.47 m and substantially increased route maneuvering. Scenario 3 showed that planner success and execution feasibility are not equivalent. Waffle A* required approximately 1.10 s of planning and produced a 240.16 s safety-aware mission, whereas 50,000-iteration RRT* required a median 401.98 s of planning to reduce median mission time to 237.16 s. For Ridgeback, RRT and RRT* succeeded in only 7 of 10 seeds within 50,000 iterations, and none of their successful raw paths satisfied the stricter continuous-clearance criterion. Even deterministic A* and Dijkstra exhibited small local post-processing clearance violations of 8.2 mm and 14.2 mm, respectively, despite feasible raw paths.

The final engineering result is therefore not a universal winning planner. It is a structured decision framework showing that route availability, planner reliability, continuous feasibility, computational cost, and mission-level benefit can diverge substantially. Under the tested assumptions, A* provides the strongest overall trade between reliability, computation, path quality, and mission performance, while RRT* demonstrates that additional optimization effort does not automatically translate into proportional operational benefit.

Contents

1	Introduction	5
2	Benchmark Philosophy and Engineering Scope	6
2.1	Established planners as evaluation inputs	6
2.2	Why a synthetic office/lab/warehouse-like environment was used	6
2.3	Why temporary and caution-required objects were added	6
2.4	Fixed conditions rather than result-driven tuning	6
3	Implementation Architecture and Algorithm-Level Validation	7
3.1	Modular benchmark structure	7
3.2	Common graph-search model	7
3.3	Independent correctness check using public benchmarks	7
3.4	Continuous-edge validation for RRT	7
3.5	RRT* rewiring and implementation correction	8
4	Preliminary Robot-Aware Metric Sanity Check	8
5	Metric Environment and Benchmark Architecture	9
5.1	STech Lab	9
5.2	Why one environment was retained for both scenarios	11
6	Robot Selection and Physical Models	11
6.1	Selection rationale	11
6.2	Configuration-space screening	12
7	Scenario Design Philosophy	12
8	Scenario 1 — Footprint-Driven Route Divergence	13
8.1	Scenario construction	13
8.2	Rectangular-footprint verification	13
8.3	LOS refinement and smoothing	15
9	Scenario 2 — Dense Temporary-Obstacle Field	17
9.1	Design intent	17
9.2	A* baseline result	17
9.3	Continuous trajectory result	18
10	Trajectory Geometry and Mission-Time Model	19
10.1	Why path length alone is insufficient	19
10.2	Waffle speed constraint	20
10.3	Ridgeback speed constraint	20
10.4	Acceleration feasibility	20
11	Code Architecture and Role of Each Module	20
12	Results Across the Two Scenarios	22
13	Scenario 3 — Safety-Aware Multi-Planner Evaluation	24
13.1	Why a third scenario was required	24
13.2	Temporary-object extraction and data lineage	24
13.3	Resolution-consistent safety allowance	25

14 Common Path-to-Execution Evaluation Pipeline	26
14.1 Three distinct notions of success	26
14.2 Quantifying clearance rather than reporting only pass/fail	27
14.3 Interpreting a clearance violation on the map	27
14.4 Why post-processing is revalidated	28
15 Human-Environment Operating-Speed Policy	28
15.1 Open-space cruise	28
15.2 Caution-zone speed cap	29
16 Final Planner Benchmark Design	30
16.1 Deterministic planners	30
16.2 Stochastic planners	30
16.3 Recorded metrics	31
17 Scenario 3 Results — TurtleBot3 Waffle Pi	31
17.1 Deterministic baselines	31
17.2 RRT variability	31
17.3 RRT* quality and computational cost	32
18 RRT* Convergence and Diminishing Returns	33
19 Scenario 3 Results — Clearpath Ridgeback	35
19.1 Deterministic planners: feasible raw paths, local refined violations	35
19.2 RRT and RRT* search difficulty	37
19.3 RRT mission quality	37
19.4 RRT* mission quality	38
20 Robot Footprint Changes Planner Search Difficulty	39
21 Planning Computation Versus Mission Benefit	39
22 Integrated Engineering Findings	40
22.1 Finding 1: robot geometry changes feasibility before planning	40
22.2 Finding 2: ordinary clutter can amplify robot-dependent route cost	40
22.3 Finding 3: planner success is not execution feasibility	40
22.4 Finding 4: path-length optimization is not safety optimization	41
22.5 Finding 5: footprint changes search difficulty itself	41
22.6 Finding 6: the best geometric path need not be the best engineering choice	41
23 Limitations and Interpretation Boundaries	41
24 Reproducibility and Data Products	42
25 Conclusion	42
Appendices	43
A Software Module Reference	43
B Development Diagnostics and Traceability	44
B.1 Safety-margin screening	44
B.2 RRT/RRT* high-budget pilot	44
B.3 Validation-method diagnostic	44
B.4 Preliminary Scenario 1 timing model	45

C	Full Scenario 3 Trial Tables	46
D	RRT* Convergence Tables	49
E	Reproducibility Commands and Data Products	49
E.1	Primary data products	49
E.2	Representative execution commands	49
E.3	Random-seed definition	50
E.4	Scenario summary	50

1 Introduction

Mobile-robot path planning is often introduced as a geometric or graph-search problem: given a start configuration, a goal configuration, and a map of obstacles, determine a collision-free path of minimum or near-minimum cost. Classical graph-search methods such as Dijkstra and A*, and sampling-based methods such as RRT and RRT*, are well suited to studying this problem at the algorithmic level. However, a path that is valid for an abstract point robot is not necessarily a useful path for a physical mobile robot operating in an indoor human environment.

A deployed robot has finite length and width, a specific drive architecture, translational and rotational limits, finite acceleration capability, and nonzero discretization and tracking uncertainty. The environment is also not composed only of permanent walls. Offices, laboratories, and warehouses contain furniture, equipment, carts, staged materials, temporary work objects, and people occupying regions that may remain free in a nominal architectural map. Practical navigation performance therefore cannot be interpreted from the planning algorithm alone.

The central premise of this study is that path planning should be evaluated as a **robot-dependent execution problem** rather than only as an isolated shortest-path problem. A useful planning result depends jointly on environment structure, robot geometry, motion capability, planner behavior, path refinement, safety assumptions, and the way the resulting trajectory is executed. A compact conceptual model is

$$\text{Navigation Performance} = f(M, F_r, A, P_{\text{post}}, C_{\text{safety}}, K_r, V_r),$$

where M denotes the metric environment, F_r the robot footprint, A the planning algorithm, P_{post} the common post-processing pipeline, C_{safety} the clearance and operational safety assumptions, K_r the robot kinematics, and V_r the velocity and acceleration constraints.

This distinction is important because several fundamentally different outcomes are easily collapsed into a single binary “planner failure.” A requested mission may be physically impossible before a planner is called. A planner may fail to discover an existing route within its computational budget. A returned path may pass the planner’s native collision test yet fail a stricter continuous-clearance check. A geometrically valid raw path may become locally infeasible after simplification or smoothing. Finally, a completely feasible trajectory may still be operationally poor if it requires excessive planning time or forces slow execution around clutter.

Accordingly, this project does **not** propose a new path-planning algorithm. Established algorithms are used as controlled inputs to a common engineering benchmark. The objective is to determine whether paths produced by those methods remain physically feasible and operationally useful once increasingly realistic robot and environment constraints are introduced.

The study is organized as a deliberate progression:

Algorithm Validation → Robot Geometry → Environmental Clutter
Execution Feasibility → Planner Trade Study

Scenario 1 isolates the effect of robot footprint on route availability. Scenario 2 introduces dense temporary clutter to determine whether local occupancy amplifies robot-dependent route cost. Scenario 3 fixes the environment and robot assumptions and compares multiple planners through a common safety-aware path-to-execution pipeline.

The final question is therefore not merely

Which planner returns the shortest path?

but rather

Can the actual robot execute the returned path safely and efficiently under the tested environmental and operational constraints?

2 Benchmark Philosophy and Engineering Scope

2.1 Established planners as evaluation inputs

Dijkstra, A*, RRT, and RRT* are treated as representative planner classes rather than research contributions. Dijkstra provides a deterministic graph-search reference without heuristic guidance. A* introduces a geometry-consistent heuristic. RRT emphasizes stochastic exploration and first-feasible-path discovery. RRT* adds iterative path-cost improvement through minimum-cost parent selection and rewiring.

The purpose of comparing them is not to restate textbook optimality properties. The benchmark instead asks how those algorithmic characteristics propagate into a physical navigation problem after robot geometry, common refinement, continuous clearance, and operating policy are applied.

2.2 Why a synthetic office/lab/warehouse-like environment was used

The metric benchmark environment is intentionally synthetic. It is not presented as a geometrically faithful reconstruction of a specific building. Its purpose is to reproduce structural characteristics commonly encountered by indoor service and logistics robots while preserving experimental control and repeatability.

The environment therefore includes long corridors, room-to-room transitions, narrow passages, bottlenecks, open regions, fixed structures, equipment-like obstacles, and more than one plausible route family. These features allow robot footprint and clutter to change route availability in interpretable ways. The map resolution of 0.05 m/cell ties those geometric features to physical robot dimensions rather than arbitrary grid units.

2.3 Why temporary and caution-required objects were added

A nominal office or warehouse map represents permanent geometry but not the full occupancy experienced during normal operation. Human-shared environments contain people, carts, trolleys, boxes, staged material, portable equipment, maintenance objects, and temporary work zones. The circular and polygonal objects added in Scenario 2 and retained in Scenario 3 are a controlled abstraction of this class of temporary or movable occupancy.

They are *not* intended to reproduce exact human shapes or predict dynamic trajectories. The benchmark does not claim dynamic-obstacle prediction or local replanning. Instead, the temporary-object layer provides a repeatable way to study route availability, obstacle clearance, cautious-speed operation, and the interaction between robot footprint and planner behavior.

This separation is deliberate. Permanent walls define the structural workspace; temporary objects modify the operational workspace. Scenario 3 further retains the temporary objects as a distinct caution layer so that proximity can influence execution speed without altering each planner’s internal objective function.

2.4 Fixed conditions rather than result-driven tuning

A central experimental principle is that conditions are not adjusted after observing planner outcomes merely to make every method succeed. Robot dimensions, safety allowance, obstacle placement, common post-processing, and evaluation criteria are held fixed across planners. If a planner fails, the failure is retained. If a refined path violates the clearance criterion, the magnitude and affected path length are recorded rather than hidden by reducing the safety requirement.

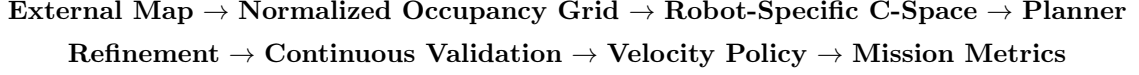
This matters because the aim is not to produce visually perfect trajectories from every algorithm. The aim is to generate evidence that can support an engineering decision.

3 Implementation Architecture and Algorithm-Level Validation

3.1 Modular benchmark structure

The project was implemented as a modular benchmark framework rather than a single demonstration script. Environment representation, robot models, planners, metrics, refinement, timing, visualization, and benchmark orchestration are separated so that changing one layer does not silently change the others.

The principal data flow is



This separation allows the same planner implementation to operate on public grid benchmarks during algorithm validation and on robot-specific metric configuration spaces during the physical study.

3.2 Common graph-search model

Dijkstra and A* use an 8-connected grid. Straight transitions have cost 1, diagonal transitions have cost $\sqrt{2}$, and diagonal corner cutting is prohibited. The shared neighbor definition ensures that differences between Dijkstra and A* arise from search strategy rather than inconsistent legal moves.

Dijkstra uses accumulated path cost $g(n)$ as its queue priority. A* uses

$$f(n) = g(n) + h(n),$$

with the octile-distance heuristic

$$h(n) = \max(\Delta x, \Delta y) + (\sqrt{2} - 1) \min(\Delta x, \Delta y),$$

which is consistent with the 8-connected motion model.

3.3 Independent correctness check using public benchmarks

Before robot geometry was introduced, MovingAI maps and reference scenarios were used as an independent correctness check. Dijkstra reproduced the published reference path lengths to numerical precision; the maximum observed absolute difference in the 100-scenario validation was approximately 1.0×10^{-8} grid-cost units with zero failures. A* produced the same optimal grid costs while expanding substantially fewer nodes. In the 30-scenario-per-map comparison, node expansion fell by approximately 72.8–87.3% depending on topology.

This phase validated map parsing, coordinate convention, graph connectivity, diagonal cost, corner-cutting prevention, and path reconstruction before any physical robot conclusions were drawn.

3.4 Continuous-edge validation for RRT

The sampling-based planners operate in continuous (x, y) coordinates over the occupancy grid. A candidate RRT extension is checked along its complete edge rather than only at its endpoint:

$$\mathbf{p}(t) = \mathbf{p}_0 + t(\mathbf{p}_1 - \mathbf{p}_0), \quad 0 \leq t \leq 1.$$

This prevents a long extension from crossing an obstacle even when both endpoints happen to lie in free cells. The RRT implementation uses goal-biased sampling, nearest-node selection, fixed-step steering, complete-edge collision checks, and first-feasible goal connection. It records path length, planning time, iterations, tree nodes, and collision checks.

3.5 RRT* rewiring and implementation correction

RRT* extends RRT through lower-cost parent selection and rewiring of nearby nodes. During the preliminary benchmark, a cost-propagation issue was identified: changing a node’s parent requires not only updating that node’s cost-to-come but recursively updating all descendants so that

$$J(c) = J(p) + d(p, c)$$

remains valid throughout the affected subtree. The implementation was corrected to propagate descendant costs and to re-evaluate goal candidates after rewiring before the final Scenario 3 experiments.

This development step is retained in the report because it demonstrates an important methodological point: planner output was not assumed correct simply because a trajectory was produced. Algorithm behavior was independently inspected before being used as evidence in the robot-aware benchmark.

4 Preliminary Robot-Aware Metric Sanity Check

Before STech Lab was constructed, the robot-model and configuration-space pipeline was exercised on a smaller metric ROS/Nav2 depot map. This intermediate stage is retained because it established an important result before any scenario was deliberately designed: robot geometry can invalidate missions before a path planner is invoked.

The preliminary depot map measured 604×307 cells at 0.05 m/cell, corresponding to approximately 30.20 m by 15.35 m. Applying the conservative circumscribed-circle footprints reduced free configuration space from 179,481 cells to 150,184 cells for Waffle and 111,032 cells for Ridgeback. The corresponding free-space losses were 16.32% and 38.14%.

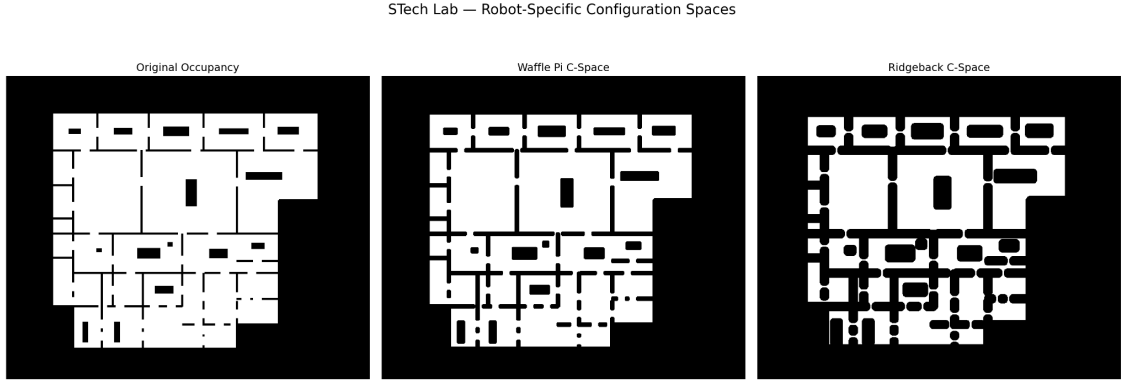


Figure 1: Preliminary metric-map configuration-space comparison. The same physical occupancy map becomes substantially more restrictive for the larger Ridgeback footprint.

A common set of 100 start-goal missions was then generated from the original metric workspace and applied unchanged to both robot-specific configuration spaces. Waffle retained 70 valid and reachable missions; Ridgeback retained only 37. Thirty-three missions were feasible for Waffle but invalid for Ridgeback, while no mission was Ridgeback-only. No planner had yet been run for this comparison.

Table 1: Preliminary common-mission feasibility on the metric depot map.

Mission status	Waffle	Ridgeback
Valid and reachable	70	37
Start invalid	9	21
Goal invalid	15	24
Both invalid	6	18
Disconnected	0	0

This intermediate result motivated the later scenario design. It showed that planner comparison should not begin by assuming a common task is physically valid for every platform. The benchmark must first separate

$$\boxed{\text{Mission Feasibility} \neq \text{Planner Success.}}$$

STech Lab was then created as a larger, more interpretable environment in which route families, bottlenecks, and temporary clutter could be controlled deliberately.

5 Metric Environment and Benchmark Architecture

5.1 STech Lab

The main benchmark uses an office/laboratory-style occupancy map named *STech Lab*. The map resolution is

$$r_{\text{map}} = 0.05 \text{ m/cell},$$

with dimensions of 1200×1000 cells, corresponding to a 60 m by 50 m metric workspace.

The environment was created to retain realistic features relevant to indoor mobile robotics:

- long open corridors in which a fast platform can exploit nominal speed;
- narrow passages and bottlenecks where footprint becomes important;
- multiple route families between common regions;
- large open areas that can be modified with temporary obstacles without changing the permanent building structure.

The environment is best interpreted as a controlled metric benchmark with an office/lab topology rather than a claim of exact reconstruction of a particular building. This distinction is deliberate: the geometry is used to create repeatable planning experiments with known physical scale.

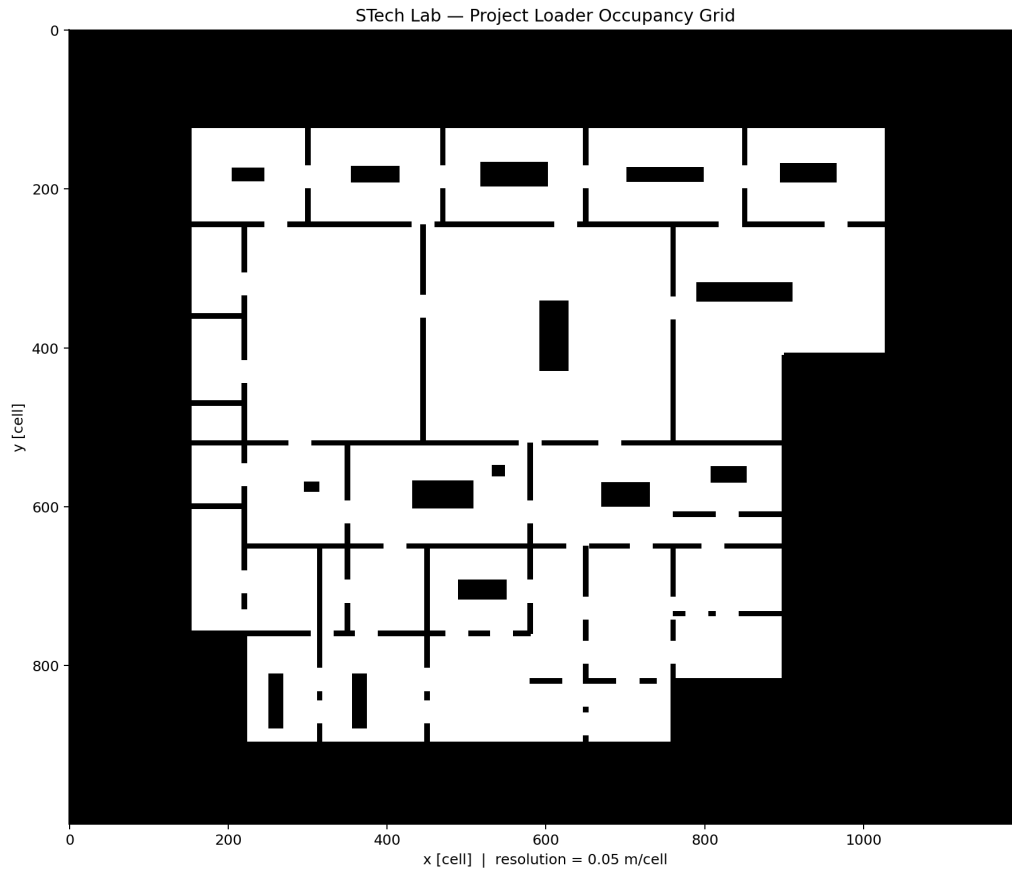


Figure 2: STech Lab occupancy grid after loading through the project metric-map pipeline. The map resolution is 0.05 m/cell and the map serves as the common structural environment for both scenarios.

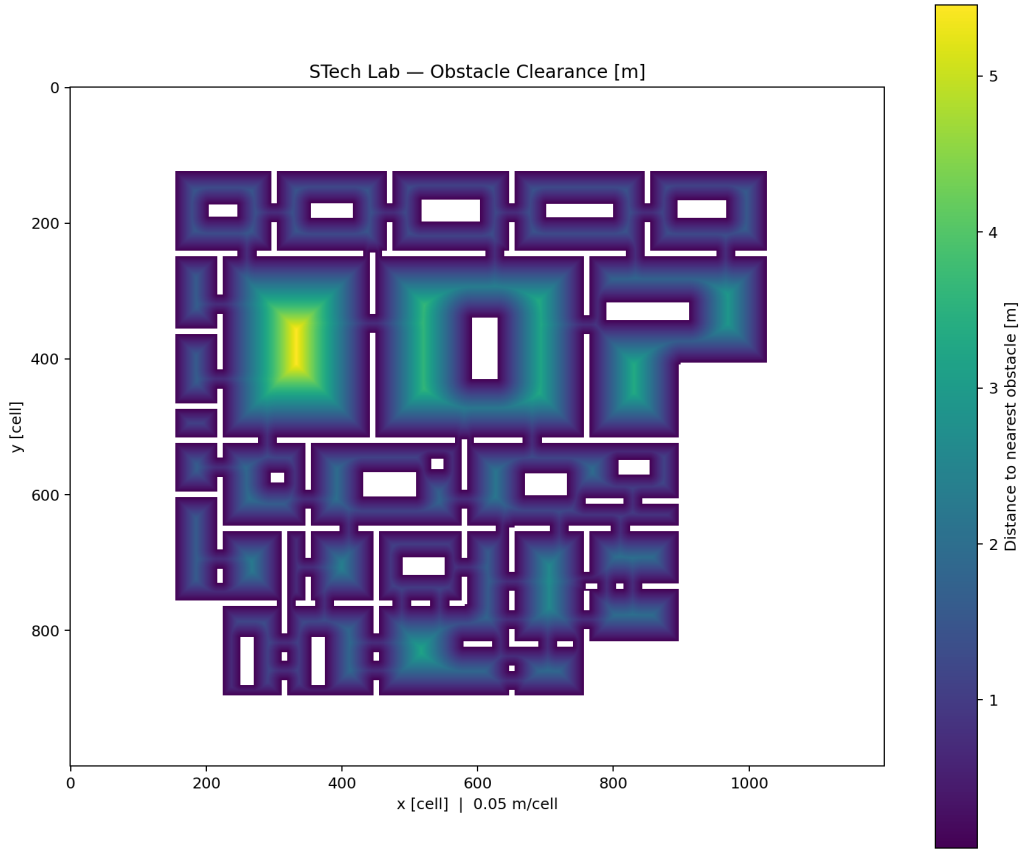


Figure 3: Euclidean obstacle-clearance field used to construct conservative robot-specific configuration spaces. Clearance is expressed in metres rather than integer dilation cells.

5.2 Why one environment was retained for both scenarios

The start and goal were fixed at

$$S = (360, 136), \quad G = (744, 840)$$

in image-cell coordinates.

Keeping the same start and goal across Scenarios 1 and 2 isolates the effect of environmental clutter. Scenario 1 uses the static STech Lab map. Scenario 2 modifies that same map with temporary obstacles while preserving the structural walls and the mission endpoints.

This avoids conflating three variables at once. The comparison between Scenario 1 and Scenario 2 therefore answers a cleaner question:

How does added local clutter change the same robot mission?

6 Robot Selection and Physical Models

6.1 Selection rationale

TurtleBot3 Waffle Pi and Clearpath Ridgeback were selected because they are not merely two differently named robots; they occupy meaningfully different parts of the mobile-robot design space.

Waffle Pi is compact and relatively slow. Ridgeback is substantially larger but has much higher nominal translational speed. This creates a useful physical trade:

small footprint / low speed vs. large footprint / high speed.

That trade is central to the benchmark. If two robots had nearly identical dimensions and velocity limits, many route differences would disappear and the experiment would reduce primarily to planner comparison.

Table 2: Robot parameters used in the benchmark.

Parameter	Waffle Pi	Ridgeback
Length [m]	0.281	0.960
Width [m]	0.306	0.793
Circumscribed radius [m]	0.2077	0.6226
Nominal $v_{\max}$ [m/s]	0.22	1.10
Longitudinal acceleration [m/s ²]	2.5	1.0
Angular speed limit [rad/s]	1.0	—
Normal-acceleration limit [m/s ²]	—	1.0

The Waffle limits correspond to the ROS 2 Humble navigation configuration used for the benchmark. Ridgeback uses a 1.1 m/s platform translational limit and a 1.0 m/s² acceleration model. The Ridgeback timing model is a translational approximation for a holonomic mecanum platform, not a wheel-level dynamic model.

6.2 Configuration-space screening

For initial planning, each rectangular footprint is converted to a conservative circumscribed circle:

$$r_c = \frac{1}{2} \sqrt{L^2 + W^2}.$$

Let $d(\mathbf{x})$ denote Euclidean distance from a free cell to the nearest occupied cell. The robot-specific feasible set is then

$$\mathcal{C}_{\text{free}} = \{\mathbf{x} \mid d(\mathbf{x}) \geq r_c\}.$$

The distance-transform formulation avoids the cell-rounding overinflation that occurred in an earlier binary-dilation implementation. It also keeps the screening method identical between robots.

This circle model is intentionally conservative. It removes heading from the global search and guarantees clearance for any orientation, but it can exclude locations that a rectangular robot could occupy at a favorable heading. For that reason, critical route claims are not based on the circle test alone.

7 Scenario Design Philosophy

The three scenarios are not independent demonstrations. They were designed as a staged engineering argument in which each scenario introduces one additional class of constraint while preserving the conclusions of the previous stage. This structure prevents planner behavior from being confused with geometry, clutter, or operating policy.

Table 3: Scenario progression and the engineering question isolated at each stage.

Scenario	Main variable	Engineering question
1	Robot footprint	Can both physical robots use the same nominal route when the environment and mission are fixed?
2	Dense temporary clutter	How strongly does operational clutter amplify robot-dependent route cost and maneuvering?
3	Planner and safety-aware execution policy	Which planner result remains useful after common refinement, continuous clearance validation, and robot-aware execution constraints?

Scenario 1 therefore establishes geometry-driven route divergence. Scenario 2 tests whether ordinary clutter magnifies that divergence. Scenario 3 then fixes the environment and robot assumptions and asks whether planner-level success survives the complete path-to-execution chain. In this way, the final planner comparison is performed only after the physical problem has already been defined.

8 Scenario 1 — Footprint-Driven Route Divergence

8.1 Scenario construction

Scenario 1 was selected from common start–goal candidates that were feasible for both robot-specific configuration spaces. The objective was not to search for the numerically largest path ratio. The desired case was one in which the route difference had an interpretable physical cause.

The selected route has the compact Waffle using a central shortcut while Ridgeback is forced toward a wider right-side route. A* was used as the deterministic baseline during scenario construction because it made the geometry of the environment–robot interaction easy to inspect before introducing stochastic planners.

The raw A* path lengths were

$$L_{W,\text{raw}} = 43.973 \text{ m}, \quad L_{R,\text{raw}} = 50.917 \text{ m},$$

giving

$$\frac{L_{R,\text{raw}}}{L_{W,\text{raw}}} = 1.1579.$$

8.2 Rectangular-footprint verification

The key concern was whether the route separation was only an artifact of circumscribed-circle inflation. The original rectangular footprints were therefore sampled along the paths.

Both robots were collision-free on their own routes. Ridgeback was then applied to the Waffle shortcut. At the tangent heading, 340 path samples collided. More importantly, at 285 sampled path locations, no collision-free Ridgeback orientation was found even when heading was searched in 5-degree increments.

This is the critical physical validation for Scenario 1. The Waffle shortcut is not merely rejected by a conservative circle model; substantial portions of the route are unavailable to the Ridgeback rectangular footprint under the tested orientation set.

The reverse test produced no Waffle collision on the Ridgeback route.

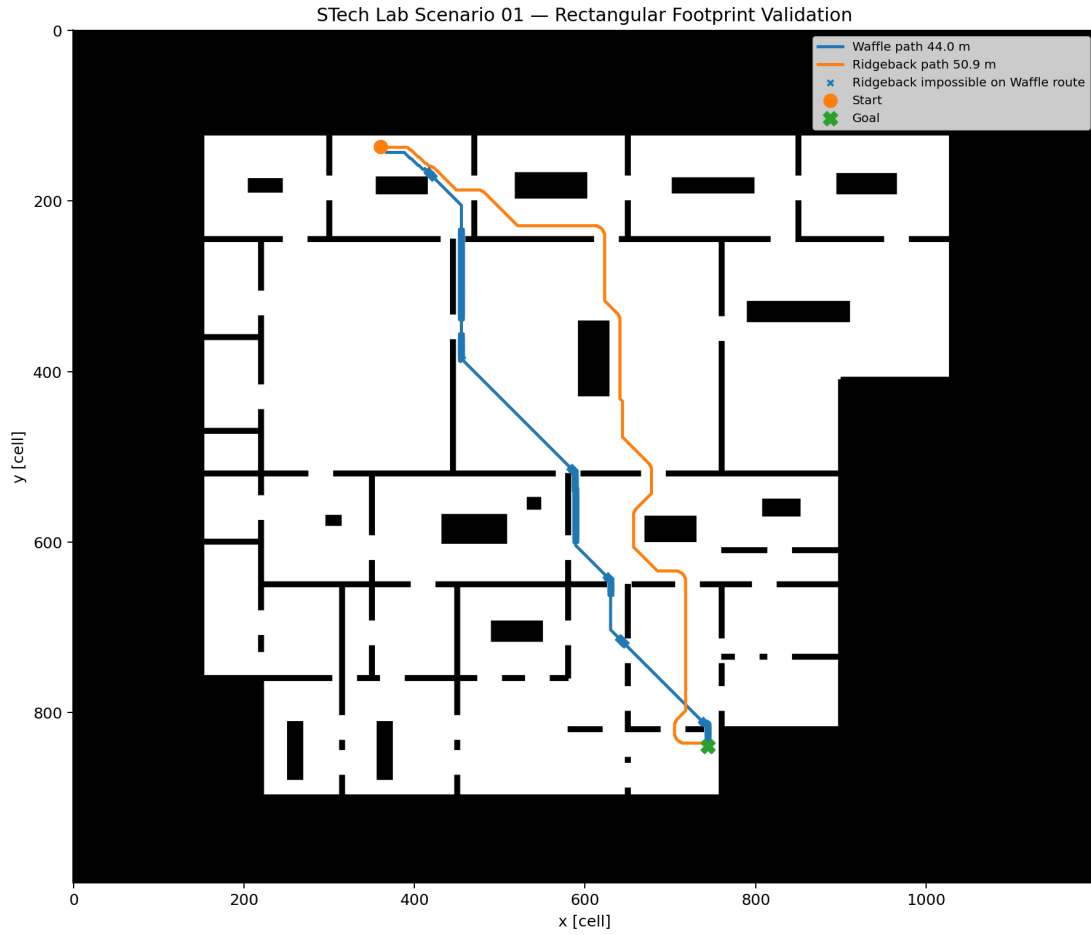


Figure 4: Scenario 1 rectangular-footprint validation. Blue marks indicate Waffle’s shortcut; the marked locations identify points where Ridgeback has no feasible sampled orientation on the Waffle route.

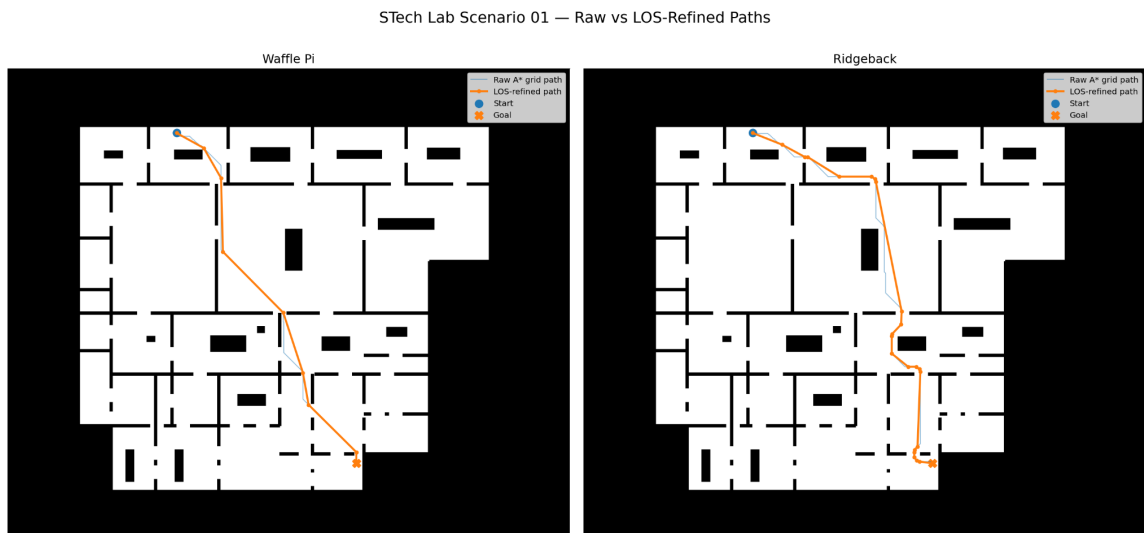


Figure 5: Scenario 1 raw A* grid paths and line-of-sight refinement. LOS removes grid staircasing before continuous geometric analysis.

8.3 LOS refinement and smoothing

Raw grid paths contain high-frequency staircase structure that should not be interpreted as commanded robot motion. A line-of-sight (LOS) simplification stage therefore removes intermediate points whenever a direct collision-free segment exists.

For Scenario 1, LOS refinement produced:

Table 4: Scenario 1 LOS path structure.

Metric	Waffle	Ridgeback
LOS path length [m]	42.584	48.787
Segments	8	23
Direction changes	7	22
Cumulative turn [deg]	218.3	618.8

Collision-aware corner smoothing was then applied, followed by uniform 0.05 m arc-length resampling. The final smoothed lengths were

$$L_W = 42.289 \text{ m}, \quad L_R = 48.566 \text{ m}.$$

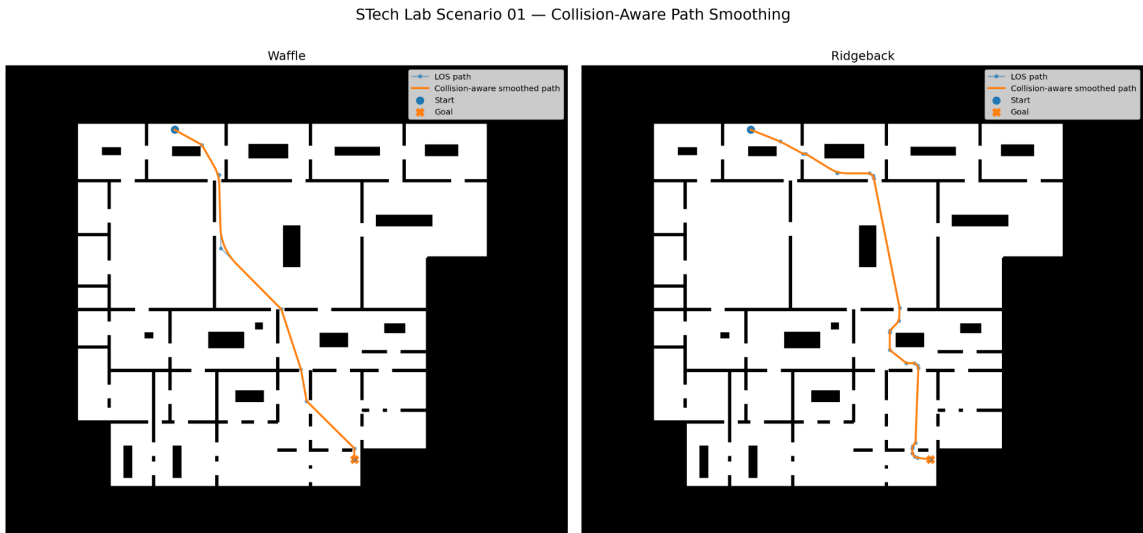


Figure 6: Scenario 1 smoothed robot-specific trajectories in STech Lab.

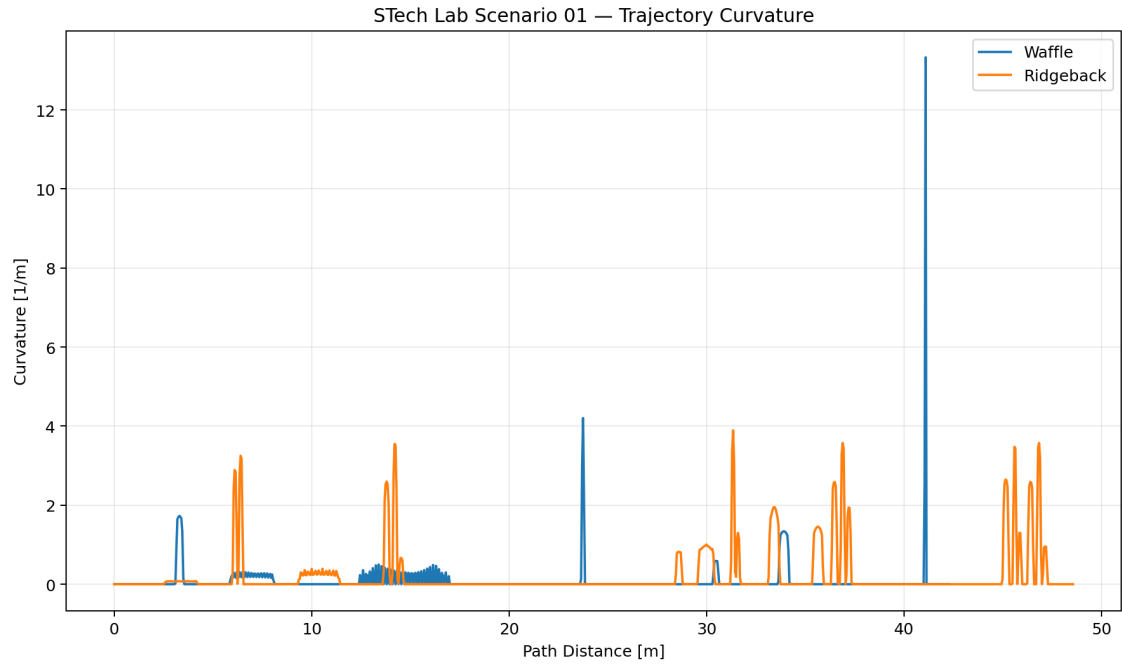


Figure 7: Scenario 1 discrete curvature after smoothing and uniform resampling. The isolated Waffle maximum is treated as a local numerical/smoothing artifact rather than a representative curvature level.

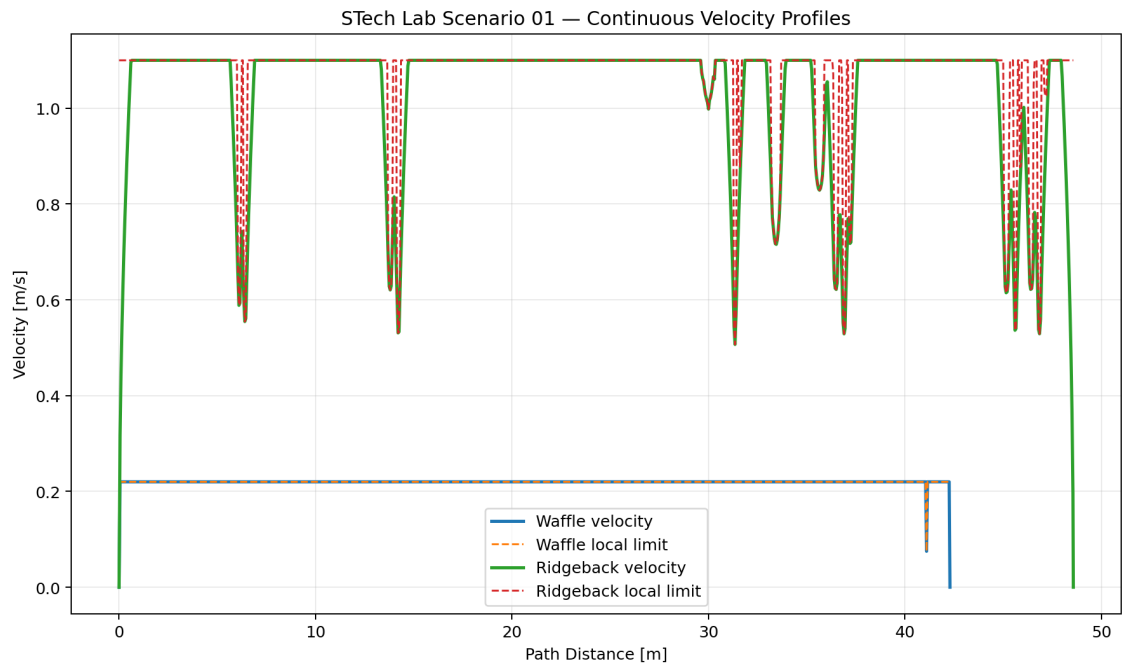


Figure 8: Scenario 1 continuous velocity profiles and local curvature-based limits. Ridgeback can exploit long high-speed sections of the static environment, while Waffle remains close to its lower nominal speed limit.

9 Scenario 2 — Dense Temporary-Obstacle Field

9.1 Design intent

Scenario 1 contains long open sections where Ridgeback can exploit its high nominal speed. That behavior is physically reasonable, but it leaves a second question unanswered:

What happens when the same mission is placed in a cluttered environment where the larger robot cannot sustain highway-like motion?

Scenario 2 was therefore created without changing the start, goal, or permanent map structure. Instead, 132 small temporary obstacles were inserted into the travel region:

- 72 circular obstacles;
- 60 pentagonal obstacles;
- nominal obstacle radius of approximately 0.20–0.35 m.

The circular and pentagonal shapes are also a visual cue. They distinguish temporary or dynamic-like objects from permanent walls and rooms. The current implementation is still a static occupancy snapshot; it does not simulate obstacle motion over time. The purpose is to represent a temporarily cluttered operating condition while retaining deterministic repeatability.

The obstacle count was deliberately high enough to break long straight corridors but not so high that the mission became infeasible. Circular configuration-space connectivity screening confirmed that both robots retained a start–goal connection.

9.2 A* baseline result

The same A* baseline produced the following route structure:

Table 5: Scenario 2 A* and LOS path results.

Metric	Waffle	Ridgeback
Raw path length [m]	44.32	83.28
LOS path length [m]	42.51	79.89
LOS waypoints	14	44
LOS segments	13	43
LOS turns	12	42
Cumulative turn [deg]	319.0	1119.4
Maximum single LOS turn [deg]	53.1	51.5

The LOS path-length ratio increased to

$$\frac{79.89}{42.51} \approx 1.88.$$

The visual result is important. Waffle remains able to use a relatively direct interior route, while Ridgeback is repeatedly displaced into larger deviations around clusters and structural boundaries.

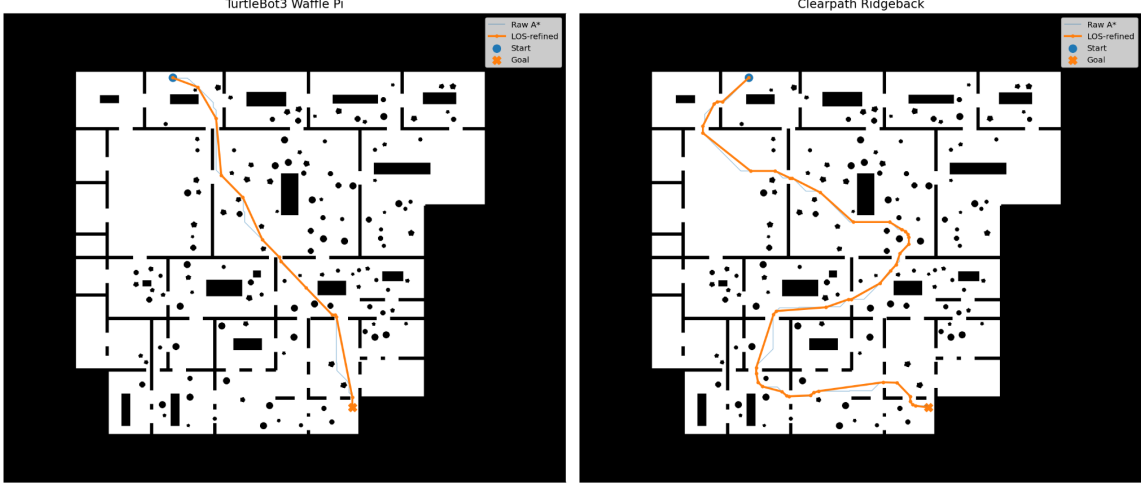


Figure 9: Scenario 2 A* comparison before final smoothing. The obstacle field changes the route topology much more strongly for Ridgeback than for Waffle.

9.3 Continuous trajectory result

After collision-aware smoothing and uniform arc-length resampling, the path lengths became

$$L_W = 42.34 \text{ m}, \quad L_R = 79.47 \text{ m}.$$

The resulting time-parameterized trajectories were:

Table 6: Scenario 2 continuous trajectory results.

Metric	Waffle	Ridgeback
Smoothed path length [m]	42.34	79.47
Peak velocity [m/s]	0.220	1.100
Minimum moving velocity [m/s]	0.142	0.201
Average velocity [m/s]	0.219	1.006
Estimated mission time [s]	193.088	79.029
Samples below 90% of $v_{\max}$	0.5%	19.6%
Samples below 75% of $v_{\max}$	0.5%	10.7%
Samples below 50% of $v_{\max}$	0.2%	0.9%

The Ridgeback still reaches 1.1 m/s where the path permits it. The important result is not that the platform can never reach maximum speed; it is that the cluttered environment reduces how continuously that speed can be exploited.

Compared with Scenario 1, Ridgeback smoothed path length increased from 48.57 m to 79.47 m, approximately 63.6%. Estimated mission time increased from 48.97 s to 79.03 s, approximately 61.4%. The Waffle values remained essentially unchanged.

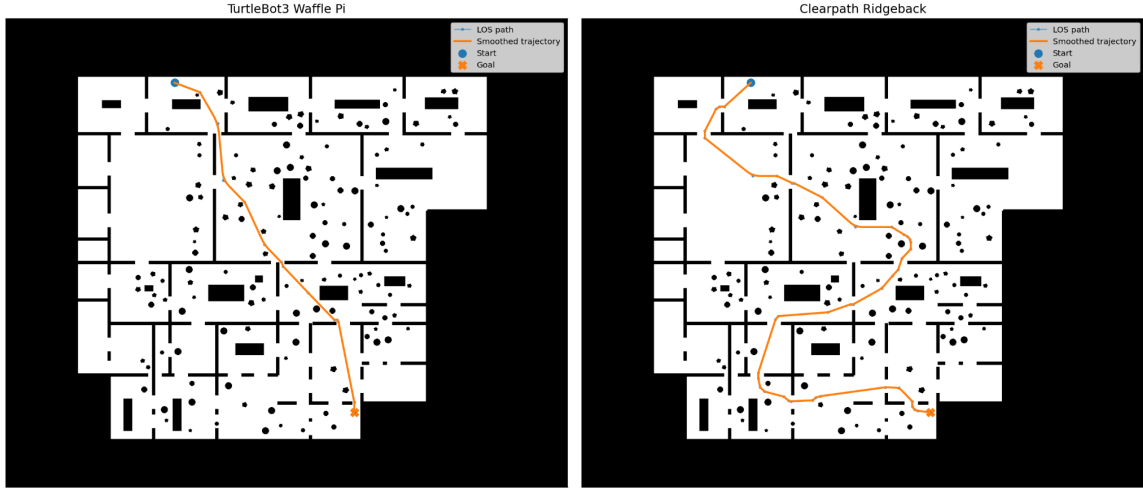


Figure 10: Scenario 2 collision-aware smoothed trajectories.

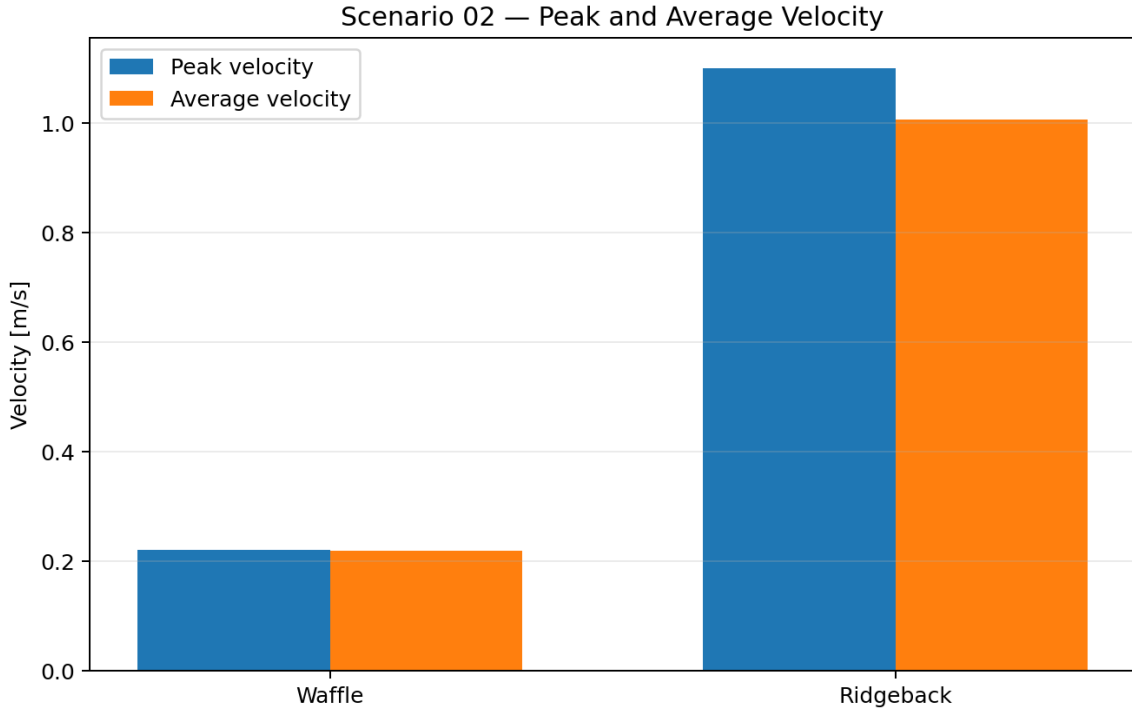


Figure 11: Scenario 2 measured peak and average translational velocity. Ridgeback still reaches its nominal peak speed, but the dense clutter reduces its sustained average.

10 Trajectory Geometry and Mission-Time Model

10.1 Why path length alone is insufficient

A planning benchmark based only on

$$J = L$$

cannot distinguish a long smooth path from a slightly shorter path with repeated high-curvature maneuvers. The benchmark therefore carries the planner output forward into a simple executable-motion approximation.

This does not claim high-fidelity vehicle dynamics. Its purpose is to expose whether the route geometry is compatible with the nominal motion capability of each platform.

10.2 Waffle speed constraint

For the differential-drive Waffle, smooth-path angular velocity is approximated by

$$\omega = v\kappa,$$

which gives the local speed limit

$$v(s) \leq \min \left(v_{\max}, \frac{\omega_{\max}}{|\kappa(s)|} \right).$$

10.3 Ridgeback speed constraint

Ridgeback is holonomic and should not be treated with the same differential-drive relation. For the current benchmark, a scalar normal-acceleration approximation is used:

$$a_n = v^2 |\kappa|,$$

giving

$$v(s) \leq \min \left(v_{\max}, \sqrt{\frac{a_{n,\max}}{|\kappa(s)|}} \right).$$

This is a trajectory-level approximation, not a mecanum wheel-force model.

10.4 Acceleration feasibility

After local curvature limits are computed, a forward and backward pass enforces longitudinal acceleration and deceleration:

$$v_{i+1} \leq \sqrt{v_i^2 + 2a_{\max}\Delta s_i}.$$

Mission time is then integrated segment by segment using the average endpoint velocity:

$$\Delta t_i = \frac{2\Delta s_i}{v_i + v_{i+1}}, \quad T = \sum_i \Delta t_i.$$

Uniform 0.05 m arc-length resampling is performed before this calculation. This step is important: using sparse LOS or smoothed points directly would allow a single long segment to hide acceleration behavior and would distort the time estimate.

11 Code Architecture and Role of Each Module

The codebase was structured so that map handling, robot feasibility, planning, refinement, timing, and visualization remain separable. This reduces the risk that a visualization or timing assumption silently changes the underlying planner.

Table 7: Primary modules used in the benchmark.

Module	Role
<code>benchmark/metric_map_loader.py</code>	Loads ROS metric occupancy maps, resolution, origin, occupancy state, and unknown-space masks. It provides the common metric map representation used by planning and RViz.

Module	Role
<code>robots/robot_model.py</code>	Stores robot geometric and motion parameters used by the benchmark.
<code>robots/footprint.py</code>	Provides footprint geometry and robot-clearance utilities. Earlier cell-dilation behavior motivated the later distance-transform screening method.
<code>robots/feasibility.py</code>	Supports collision and feasibility evaluation for robot-specific paths.
<code>planners/dijkstra.py</code>	Reference graph search used as an optimal grid-cost baseline.
<code>planners/astar.py</code>	Heuristic graph search used for deterministic baseline routes and scenario construction.
<code>planners/rrt.py</code>	Sampling-based feasible-path planner for the later stochastic benchmark.
<code>planners/rrt_star.py</code>	Asymptotically improving sampling-based planner. Rewiring and descendant-cost propagation were corrected during the preliminary validation phase.
<code>benchmark/find_robot_dependent_scenarios.py</code>	Searches common start-goal pairs on robot-specific configuration spaces and identifies candidates where platform geometry produces meaningful route divergence.
<code>benchmark/validate_scenario01_footprints.py</code>	Applies the true rectangular footprints to Scenario 1 paths and tests tangent-heading collision as well as discrete orientation feasibility. This verifies that the Waffle shortcut is not only a circular-inflation artifact.
<code>benchmark/refine_scenario01_paths.py</code>	Performs line-of-sight simplification and reports refined path length, segments, turns, and cumulative direction change.
<code>benchmark/smooth_scenario01_paths.py</code>	Applies collision-aware corner smoothing and produces continuous path geometry for curvature analysis.
<code>benchmark/scenario01_continuous_time.py</code>	Uniformly resamples Scenario 1 paths, evaluates curvature-dependent speed limits, performs forward/backward acceleration passes, and integrates mission time.
<code>benchmark/run_scenario02_astar.py</code>	Generates the Scenario 2 A* robot comparison and LOS path metrics on the dense temporary-obstacle map.
<code>benchmark/run_scenario02_analysis.py</code>	Runs Scenario 2 smoothing, uniform resampling, curvature evaluation, velocity parameterization, mission-time integration, figures, and CSV export.
<code>benchmark/scenario01_rviz.py</code>	Publishes the static map, robot-specific paths, metrics, start/goal markers, and time-profile-driven robot animation for Scenario 1.
<code>benchmark/scenario02_rviz.py</code>	Provides the equivalent ROS 2/RViz demonstration for the dense temporary-obstacle scenario.

12 Results Across the Two Scenarios

Table 8: Cross-scenario comparison using the current A* baseline and common trajectory model.

Metric	Scenario 1		Scenario 2	
	Waffle	Ridgeback	Waffle	Ridgeback
Smoothed length [m]	42.29	48.57	42.34	79.47
Mission time [s]	192.86	48.97	193.09	79.03
Ridge/Waffle length ratio	1.148		1.877	
Ridge/Waffle time ratio	0.254		0.409	

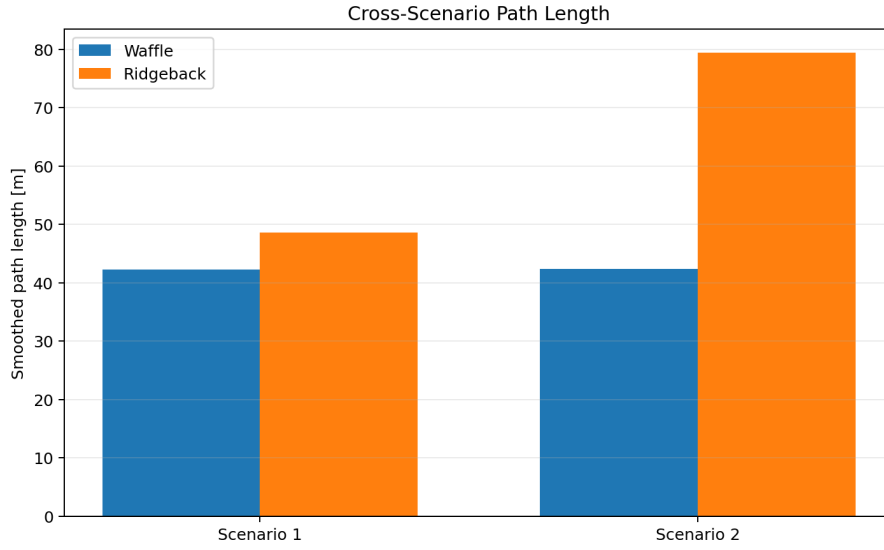


Figure 12: Cross-scenario smoothed path length. Temporary clutter leaves the Waffle route nearly unchanged but strongly lengthens the Ridgeback route.

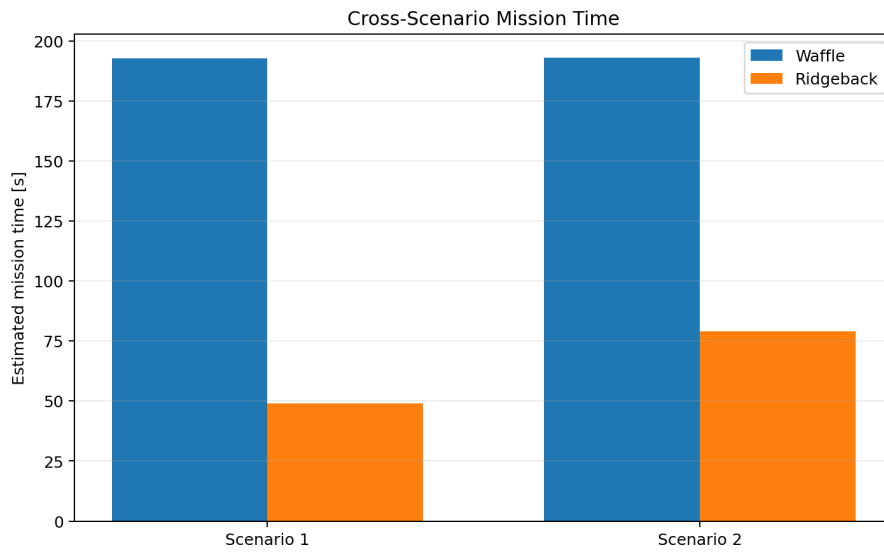


Figure 13: Cross-scenario estimated mission time under the common continuous kinematic timing model.

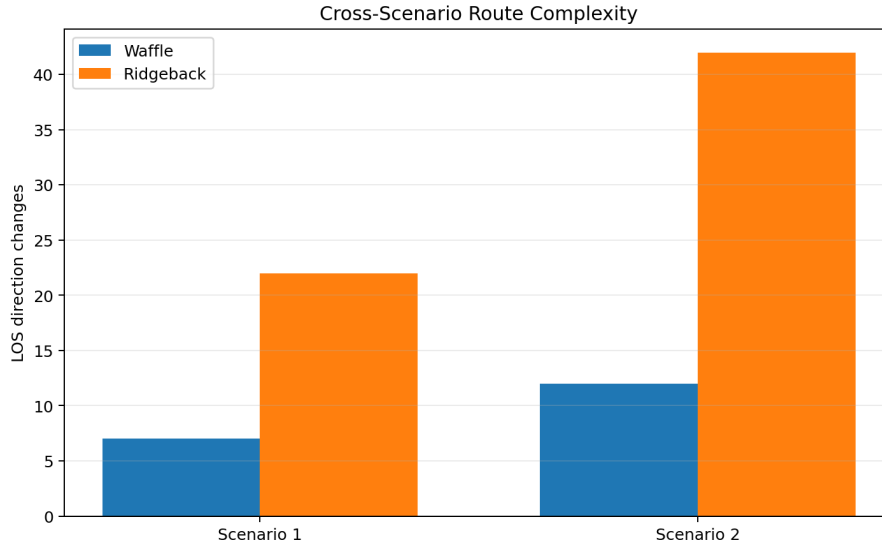


Figure 14: Cross-scenario LOS route complexity. Scenario 2 disproportionately increases Ridgeback direction changes.

The two scenarios answer different questions.

Scenario 1: footprint controls route access. The smaller Waffle can exploit a shortcut that is physically unavailable to Ridgeback. Nevertheless, Ridgeback completes the estimated mission much faster because the static environment still provides long sections where its higher nominal velocity dominates.

Scenario 2: clutter amplifies platform-dependent maneuver cost. Temporary obstacles barely change the Waffle mission, but they strongly increase Ridgeback path length and maneuver count. Ridgeback remains faster in absolute time, but its relative time advantage falls substantially.

The ratio

$$\frac{T_W}{T_R}$$

changes from approximately 3.94 in Scenario 1 to 2.44 in Scenario 2. In other words, the environment has not erased Ridgeback’s speed advantage, but it has reduced the fraction of that nominal capability that is useful for the mission.

This is the intended benchmark behavior. It demonstrates that robot comparison cannot be separated from environment structure.

13 Scenario 3 — Safety-Aware Multi-Planner Evaluation

13.1 Why a third scenario was required

Scenarios 1 and 2 established two physical facts before planner comparison. First, robot footprint can change which routes are available. Second, temporary clutter can amplify that effect dramatically. Scenario 3 asks the next question: once environment and robot assumptions are fixed, how do different planners behave when their outputs are passed through the same execution-oriented evaluation chain?

The purpose is therefore not to create a harder map for its own sake. Scenario 3 converts the dense Scenario 2 environment into a controlled planner trade study by adding a common safety allowance, retaining temporary objects as a caution layer, applying identical post-processing, validating clearance continuously, and estimating mission time under a common operating-speed policy.

13.2 Temporary-object extraction and data lineage

The dense map was compared against the underlying structural map to isolate occupancy introduced specifically as temporary clutter. The difference contained 13,299 added occupied pixels, corresponding to 33.248 m² of rasterized area at 0.05 m/cell, and exactly 132 connected components. No base-map occupied pixels were removed.

The extracted layer is important because it preserves the distinction between permanent structure and operational clutter. The planner sees both as occupied geometry, but the execution policy can treat temporary-object proximity separately.

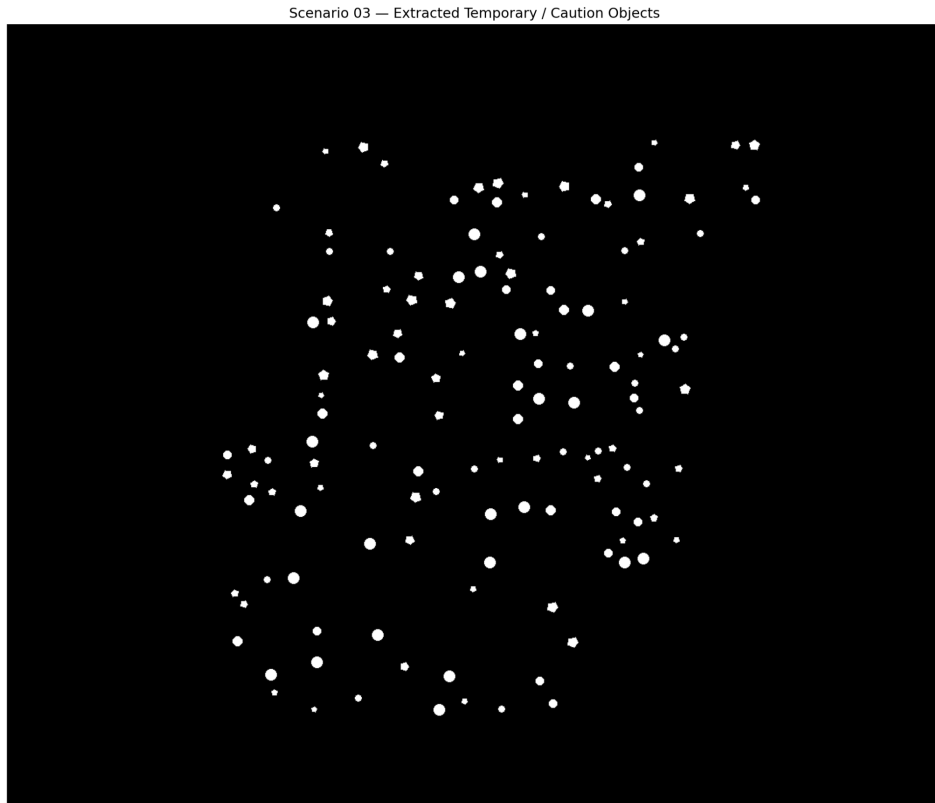


Figure 15: Scenario 3 extracted temporary/caution-object mask. The 132 connected components are derived by differencing the dense operational map from the structural base map.

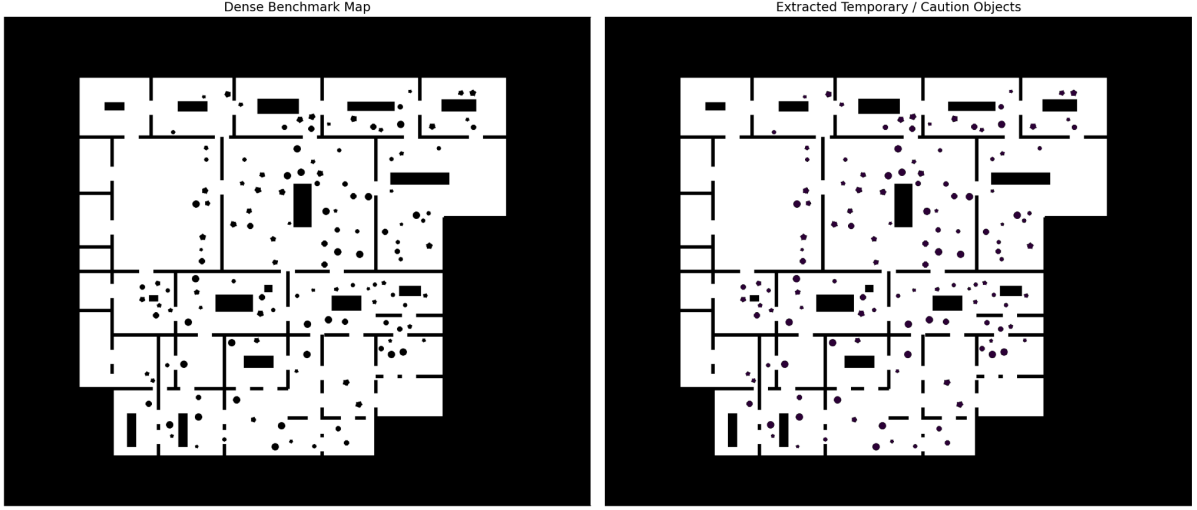


Figure 16: Dense benchmark map with the extracted temporary/caution objects highlighted. The layer represents repeatable operational clutter rather than predicted dynamic trajectories.

13.3 Resolution-consistent safety allowance

Scenario 3 adds a common geometric allowance of

$$r_{\text{safety}} = 0.025 \text{ m}$$

to each robot’s conservative footprint. The effective clearance radius is

$$r_{\text{eff}} = r_{\text{footprint}} + r_{\text{safety}}.$$

For Waffle, this gives approximately 0.233 m; for Ridgeback, approximately 0.648 m.

The value 0.025 m is not presented as an industry safety standard or calibrated sensor-error model. It corresponds to half of the 0.05 m map resolution per side, giving approximately one grid cell of total lateral allowance across opposite sides. Larger preliminary margins quickly eliminated Ridgeback start/goal feasibility in the dense map. The 0.025 m allowance was therefore adopted as a small, explicit, resolution-consistent condition that retained the common mission without being planner-specific.

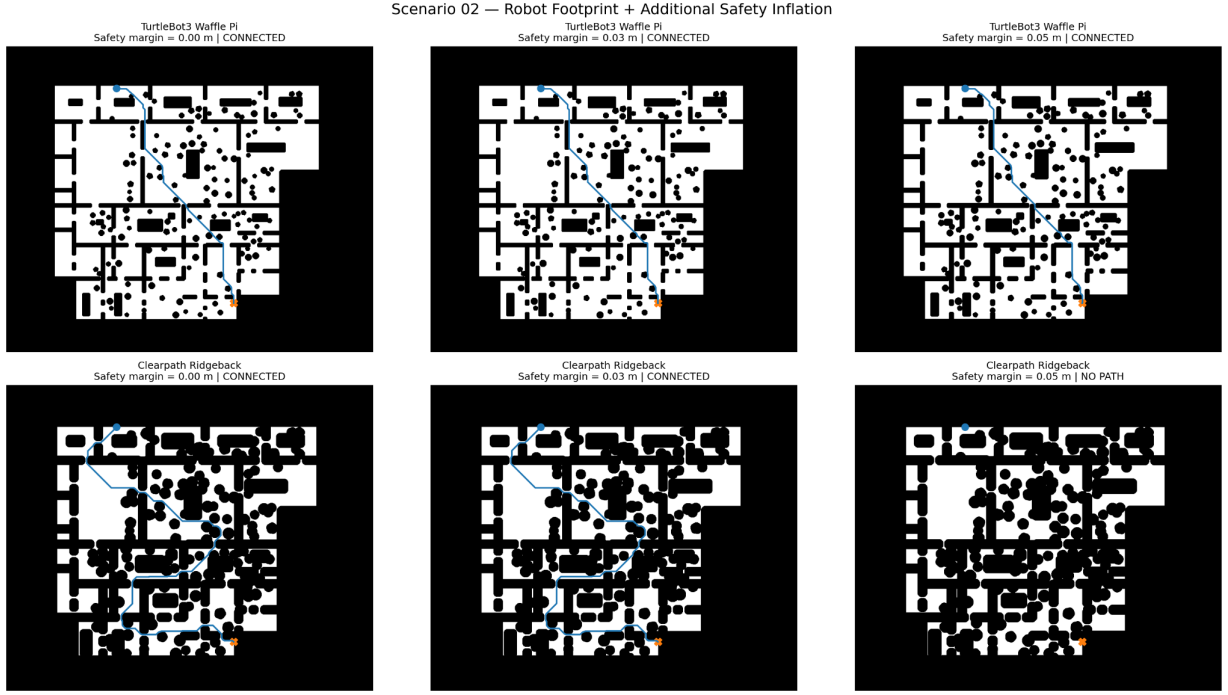


Figure 17: Safety-inflation screening illustrating how small additional margins erode free configuration space, particularly for Ridgeback. The final Scenario 3 allowance was fixed at 0.025 m rather than tuned separately by planner.

14 Common Path-to-Execution Evaluation Pipeline

The central methodological element of Scenario 3 is a planner-independent evaluation chain:

Metric Map → Robot-Specific C-Space → Planner → Raw Path
 → LOS Simplification → Smoothing → Uniform Resampling
 → Continuous Clearance Validation → Motion Limits → Operating-Speed Policy
 → Mission Time.

The same line-of-sight simplification, smoothing, resampling, clearance test, and mission-time model are applied to every planner. This prevents a planner from receiving a private post-processing advantage.

14.1 Three distinct notions of success

Scenario 3 explicitly separates three stages:

1. **Planner success:** the planner discovers a path under its own state-space and collision representation within the allotted budget.
2. **Raw continuous feasibility:** the returned raw polyline satisfies the common continuous clearance-field criterion after uniform resampling.
3. **Final trajectory feasibility:** the LOS/smoothed/resampled trajectory still satisfies the same continuous criterion.

These quantities must not be conflated. Sampling-based planners use continuous edges but still query a discretized inflated occupancy grid. The later clearance validator instead samples the Euclidean clearance field and compares it to the effective robot envelope. Therefore a planner may return a valid path under its native collision model while the stricter continuous check records a small margin violation.

14.2 Quantifying clearance rather than reporting only pass/fail

Let $d(s)$ denote the distance from the trajectory center at path coordinate s to the nearest occupied region. The available safety margin is

$$m(s) = d(s) - r_{\text{eff}}.$$

A trajectory satisfies the common clearance criterion if

$$m(s) \geq 0$$

for all sampled positions. When the condition is violated, the maximum penetration into the forbidden effective-envelope region is

$$\delta_{\text{max}} = \max \left(0, -\min_s m(s) \right).$$

The benchmark records maximum violation depth in millimetres, number of violating samples, and violation-affected sampled path length. This is more informative than a binary infeasible label. An 8 mm local margin shortfall and a deep obstacle intersection are both formally infeasible under a fixed criterion, but they represent very different geometric conditions.

14.3 Interpreting a clearance violation on the map

A violation marker represents the *robot center* at a sampled trajectory location, not the physical obstacle location. This distinction is easy to miss when only the raw occupancy map is shown. For Ridgeback, the relevant exclusion region is the obstacle set expanded by the robot footprint and the additional Scenario 3 safety allowance of 0.025 m. A centerline sample can therefore appear to lie in visually open space while the required robot envelope already overlaps the inflated forbidden region.

Figure 18 shows this interpretation side by side. The left panel presents the final path and violation samples on the nominal occupancy map. The right panel adds a translucent forbidden-region overlay to make the configuration-space interpretation explicit. The overlay is explanatory: all numerical pass/fail decisions in the benchmark are computed from the clearance field and the effective radius, not from the rendered pixels of this illustration.

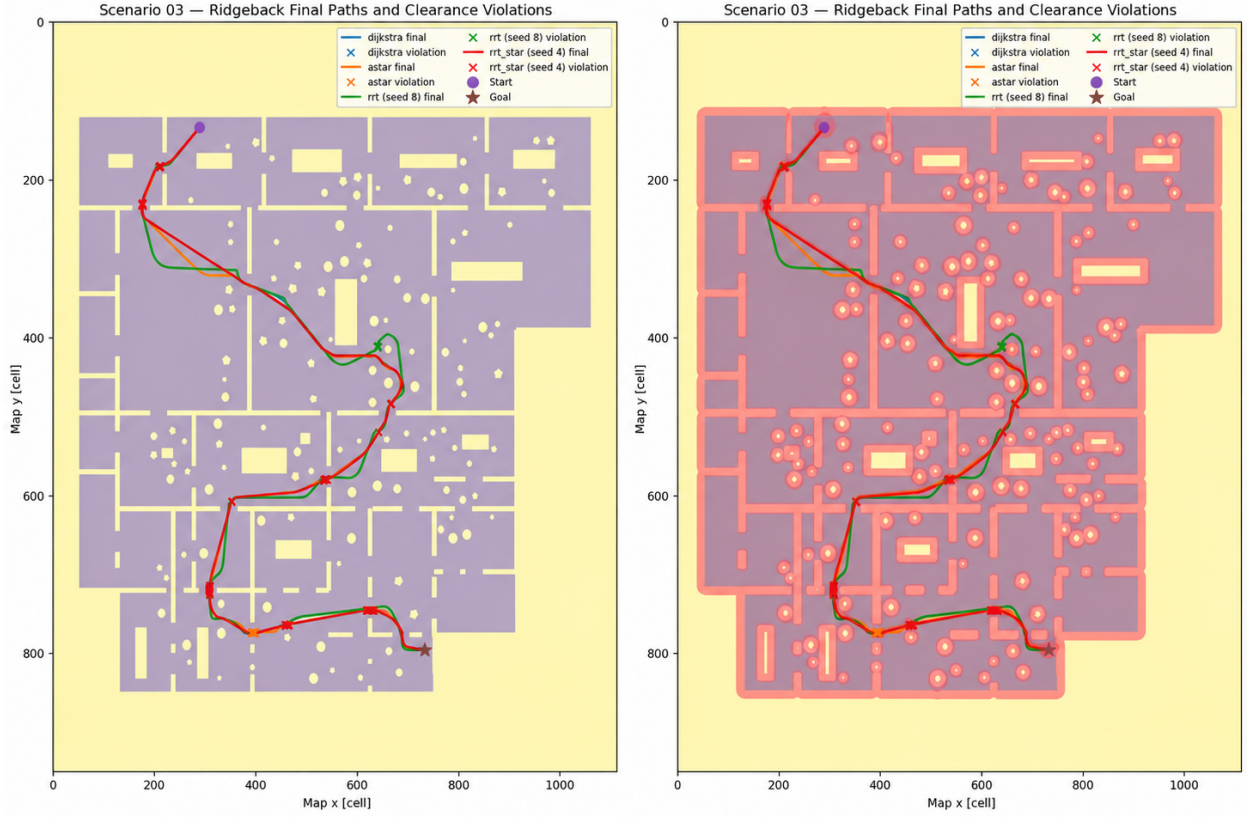


Figure 18: Interpretation of Ridgeback clearance violations. Left: robot-center trajectories and violation samples on the nominal map. Right: the same result shown with an explanatory inflated forbidden-region overlay. A violation sample does not need to lie on a raw obstacle pixel; it is infeasible when the robot center enters the region excluded by the physical footprint plus safety allowance. The benchmark classification itself is computed numerically from the continuous clearance field.

14.4 Why post-processing is revalidated

Raw planner paths are not treated as final robot commands. Grid paths contain staircase structure, and sampling-based paths contain unnecessary tree-induced turns. A common LOS simplification and collision-aware smoothing stage therefore removes redundant geometry before trajectory analysis.

However, geometric refinement is not assumed safe merely because the original path was valid. The final path is rechecked continuously. Scenario 3 demonstrates why this matters: Ridgeback A* and Dijkstra both produced feasible raw paths, yet the refined trajectories exhibited small local safety-margin violations. The benchmark retained those violations rather than retuning the smoothing or safety allowance after observing the result.

15 Human-Environment Operating-Speed Policy

15.1 Open-space cruise

Running every platform continuously at manufacturer maximum speed would not represent the intended shared indoor operating condition. Scenario 3 therefore defines open-space cruise as

$$v_{\text{open}} = 0.8v_{\text{max}}.$$

The resulting nominal operating speeds are

$$v_{\text{open,W}} = 0.176 \text{ m/s}, \quad v_{\text{open,R}} = 0.88 \text{ m/s}.$$

The 80% value is a benchmark operating policy, not a universal industry standard.

15.2 Caution-zone speed cap

A path sample is treated as being in the caution zone when the available clearance outside the effective robot envelope to the extracted temporary-object layer is less than 1.0 m. Inside this zone,

$$v_{\text{caution}} = \min(v_{\text{open}}, 0.30 \text{ m/s}).$$

The Waffle’s 0.176 m/s cruise is already below the cap, so its speed is not further reduced. Ridgeback is reduced from 0.88 m/s to 0.30 m/s near caution objects.

This policy is applied *after* planning. It is not embedded as a soft clearance cost in Dijkstra, A*, RRT, or RRT*. That separation preserves the native planner behavior while subjecting all returned paths to the same operational rule.

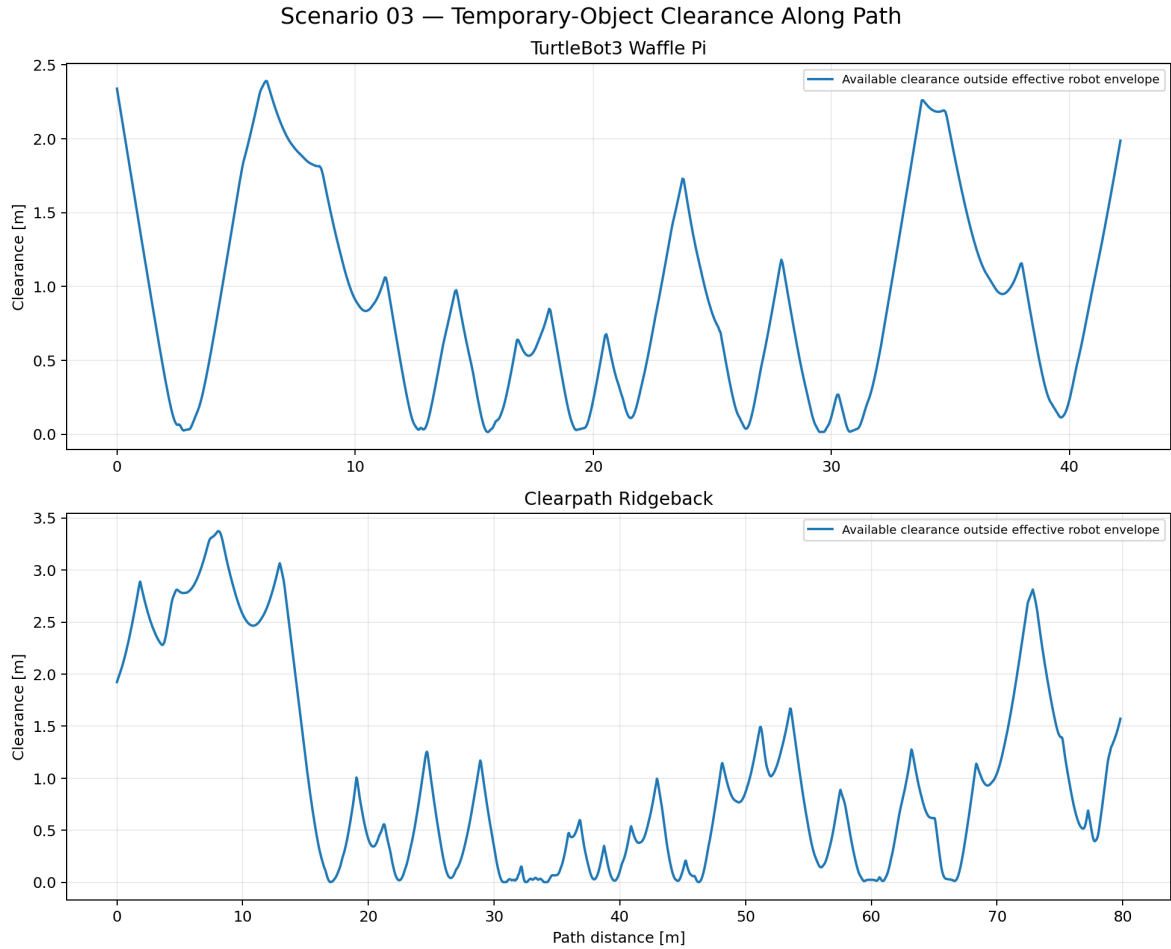


Figure 19: Available clearance outside each effective robot envelope along the representative Scenario 3 trajectory. Low-clearance regions define where temporary-object proximity becomes operationally relevant.

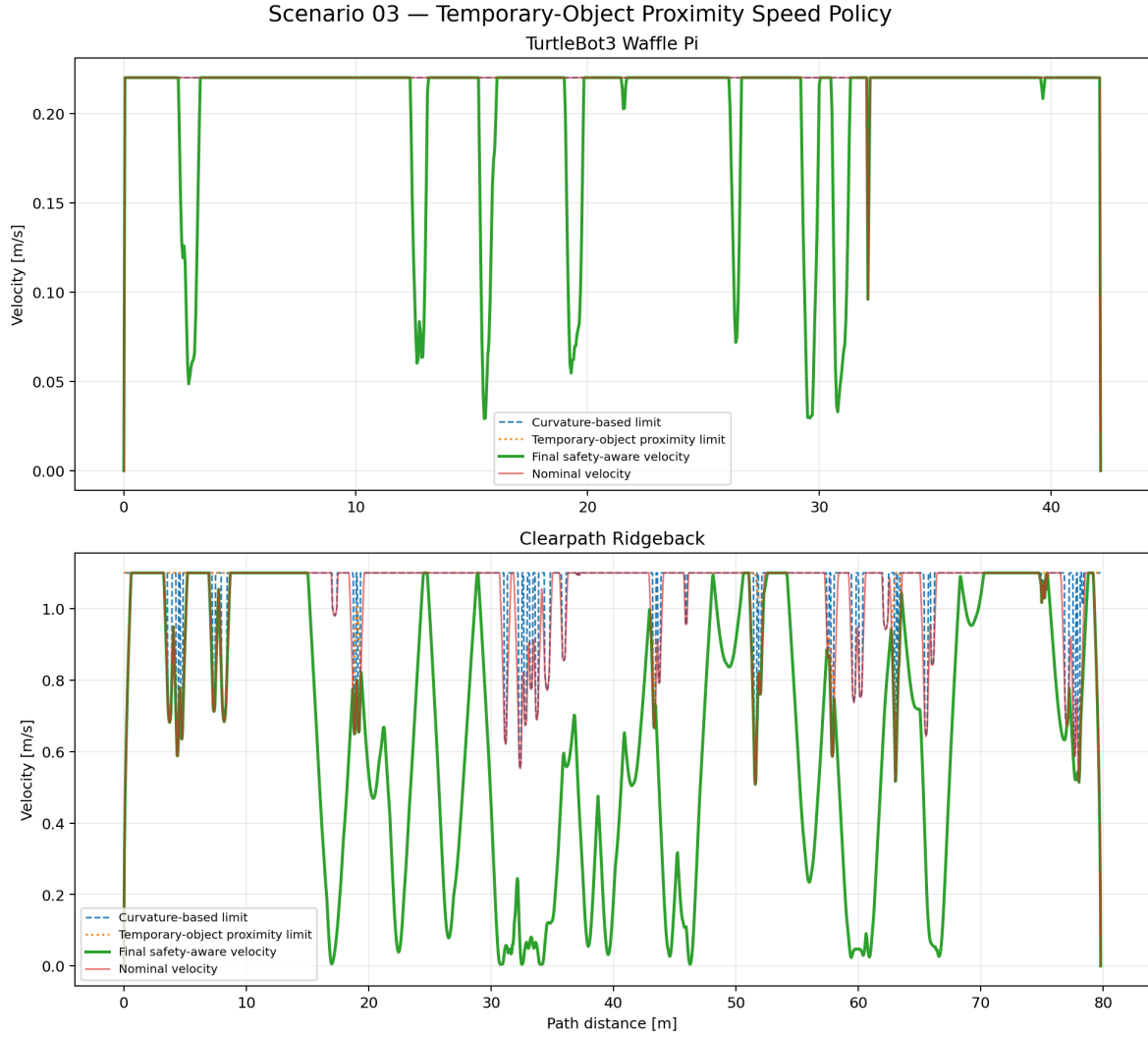


Figure 20: Scenario 3 operating-speed policy. Waffle remains at its 0.176 m/s open-space cruise because that value is already below the caution cap. Ridgeback alternates between 0.88 m/s open-space operation and the 0.30 m/s caution-zone cap, with curvature/acceleration limits applied concurrently.

16 Final Planner Benchmark Design

16.1 Deterministic planners

Dijkstra and A* are deterministic under the fixed map and cost model and were therefore evaluated once per robot. Their common 8-connected graph means that differences should primarily appear in search effort and tie-dependent path structure rather than in nominal grid-path cost.

16.2 Stochastic planners

RRT and RRT* were evaluated with seeds 0 through 9. The maximum iteration budget was fixed at 50,000 for both planners. RRT terminates as soon as it obtains a feasible goal connection; its measured planning time is therefore a first-feasible planning time. RRT* continues to the full budget so that later rewiring can improve path cost.

The RRT* convergence history was sampled at

$$N \in \{5000, 10000, 20000, 50000\}.$$

The 50,000-iteration pilot was intentionally retained even though it was expensive. The computational inefficiency itself is part of the engineering evidence needed to determine whether a smaller budget would be justified.

16.3 Recorded metrics

The final benchmark records three levels of evidence.

Planner-level metrics. Planner success, planning time, expanded nodes for graph planners, and iterations, tree nodes, collision checks, rewires, and first-solution iteration for sampling planners.

Path-level metrics. Raw path length, final path length, raw continuous feasibility, final feasibility, minimum clearance, maximum violation depth, violating-sample count, violation-affected path length, and caution-zone distance ratio.

Mission-level metrics. Nominal mission time under the open-space policy, safety-aware mission time after the caution-zone cap, and additional time introduced by the operating policy.

17 Scenario 3 Results — TurtleBot3 Waffle Pi

17.1 Deterministic baselines

Dijkstra and A* produced essentially the same final route quality. Dijkstra required 7.55 s of planning and produced a 42.13 m final path with a 240.08 s mission. A* required only 1.10 s, produced a 42.14 m final path, and yielded a 240.16 s mission. Both raw and final trajectories passed the continuous clearance criterion with zero measured violation.

The practical path-quality difference is negligible, while A* is approximately 6.9 times faster in planning for this mission. This is the expected value of an informative heuristic in a structured indoor map: search effort is reduced without changing the useful route family.

17.2 RRT variability

RRT found a planner-level solution in all 10 Waffle seeds. The median first-feasible iteration was 5,126, and median planning time was approximately 1.00 s. However, first-feasible path quality was highly seed-dependent. The median final path length was 53.10 m and the median mission time was 302.15 s, substantially worse than the deterministic baselines.

Only 1 of 10 successful raw RRT paths passed the stricter continuous-clearance check. After common LOS/smoothing, 7 of 10 final trajectories were feasible. This apparently counterintuitive improvement is possible because the common refinement can remove tree-induced detours that skim the discretized C-space boundary. The result demonstrates that post-processing is neither uniformly beneficial nor uniformly harmful; it must be revalidated explicitly.



Figure 21: Waffle RRT seed-to-seed variability across the ten final-benchmark seeds. First-feasible planning is fast, but route structure and mission quality vary substantially.

17.3 RRT* quality and computational cost

RRT* succeeded in all 10 Waffle seeds and produced a median final path length of 41.657 m and median mission time of 237.158 s. Nine of the ten raw and final trajectories passed the continuous-clearance criterion. The single failing seed had a maximum final margin violation of approximately 6.0 mm.

The path quality is excellent, but the computational cost is large. Median planning time was 401.98 s. Relative to A*, RRT* reduced median final path length by approximately 0.482 m and median mission time by approximately 3.00 s, while requiring approximately 365 times the planning computation.

For an offline path that is reused many times, that trade may be acceptable. For online task-level planning, the result provides little evidence that the extra computation is justified under the tested environment and operating policy.

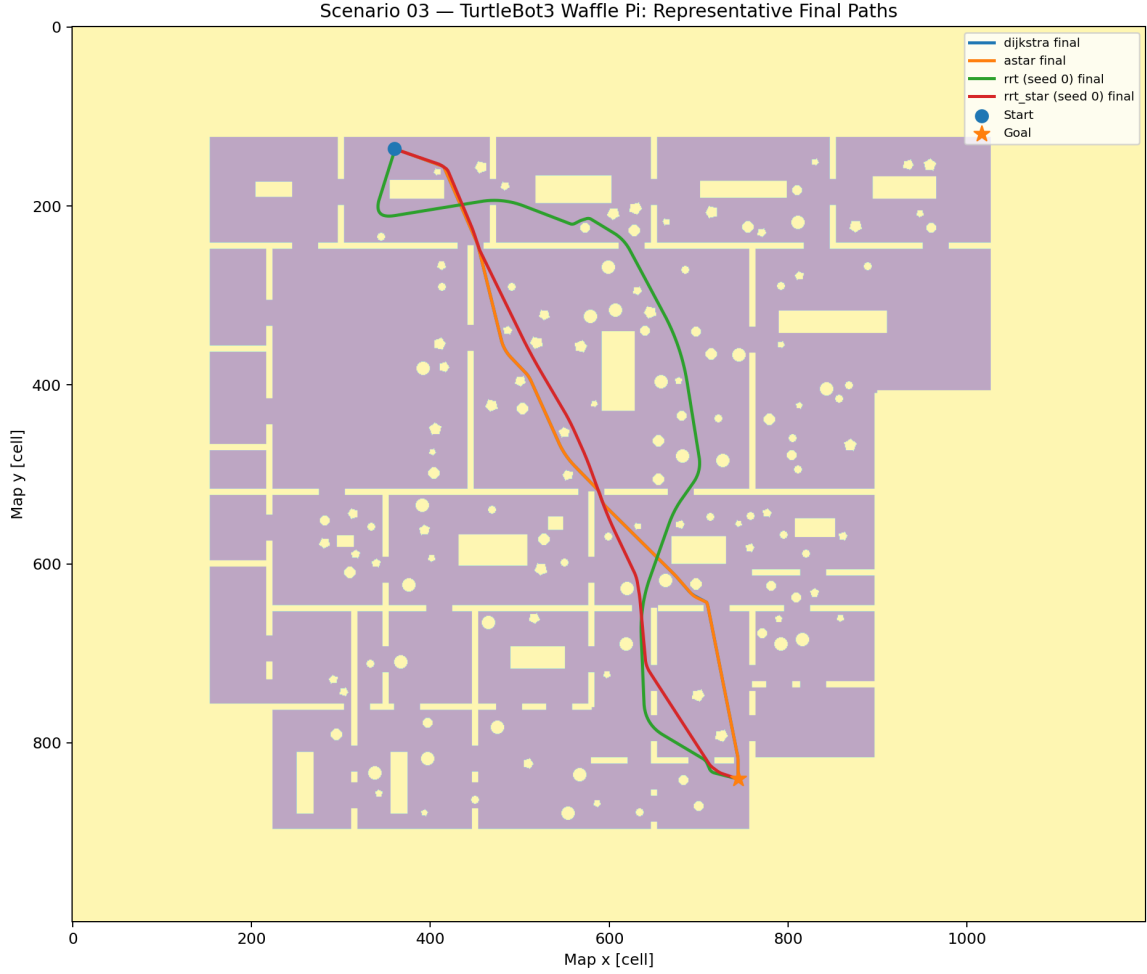


Figure 22: Representative Waffle final paths. The RRT* result is geometrically competitive with the deterministic baselines, while the representative first-feasible RRT route is visibly less direct.

18 RRT* Convergence and Diminishing Returns

Across Waffle seeds, solution availability increased from 5/10 at 5,000 iterations to 9/10 at 10,000 and 10/10 at 20,000. The median best raw goal cost improved from 50.77 m among available 5,000-iteration solutions to 44.25 m at 10,000, 41.97 m at 20,000, and 41.72 m at 50,000.

The 20,000-to-50,000 improvement is therefore only about 0.25 m in median raw path cost despite an additional 30,000 iterations. The representative seed 0 rerun shows the same pattern more directly: no solution at 5,000; 45.03 m final path and 256.27 s mission at 10,000; 41.75 m and 237.77 s at 20,000; and 41.67 m and 237.17 s at 50,000. The final 30,000 iterations improve the representative mission by only about 0.6 s.



Figure 23: Representative Waffle RRT* path convergence for seed 0. The major qualitative improvement occurs by 20,000 iterations; the 50,000-iteration route is only marginally better.

Scenario 03 — RRT* Convergence Across 10 Seeds

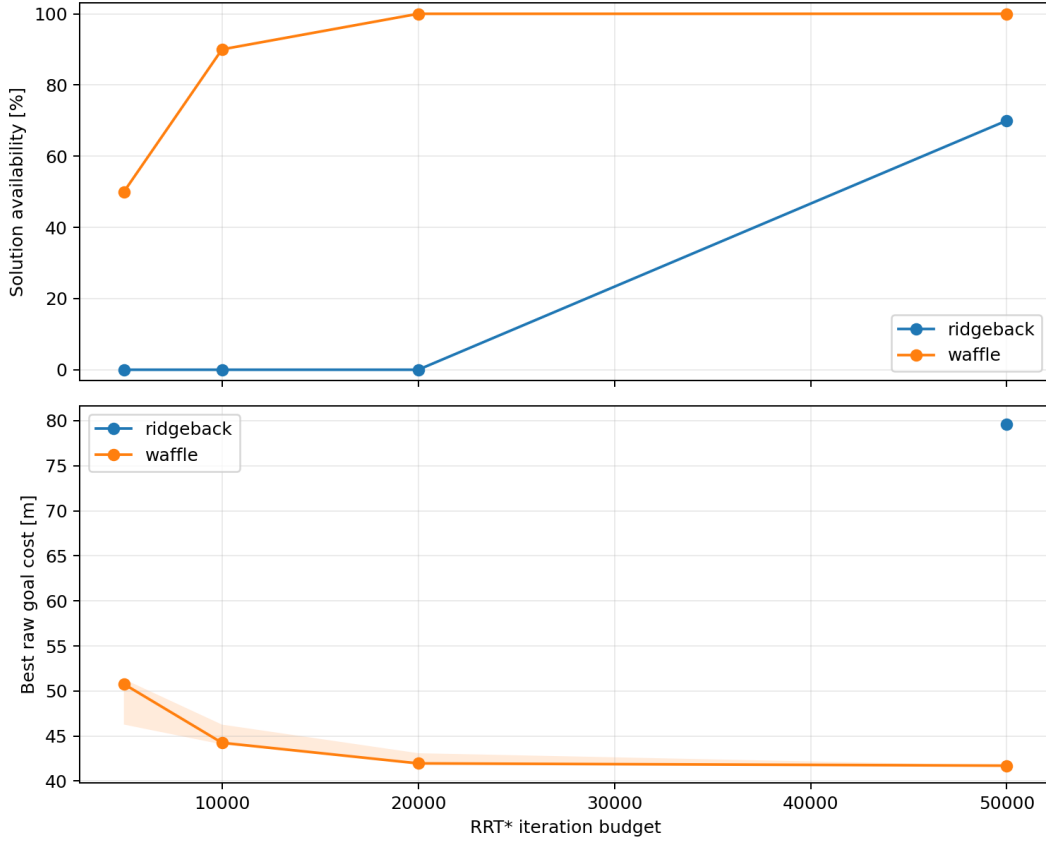


Figure 24: RRT* solution availability and best raw goal-cost statistics across ten seeds. Waffle reaches 100% availability by 20,000 iterations and then shows diminishing cost improvement. Ridgeback produces no solution in any seed through 20,000 iterations and reaches only 70% availability at 50,000.

19 Scenario 3 Results — Clearpath Ridgeback

19.1 Deterministic planners: feasible raw paths, local refined violations

Ridgeback Dijkstra and A* both succeeded at the planner level and both returned raw paths that passed the continuous-clearance criterion. Their final routes were also geometrically similar: 79.41 m for Dijkstra and 79.81 m for A*.

However, after the common refinement pipeline, both exhibited small local safety-margin violations. Dijkstra reached a maximum final violation of 14.2 mm over approximately 1.65 m of violation-affected sampled path. A* reached 8.2 mm over approximately 1.05 m. The correct interpretation is therefore not that the global route did not exist. The route existed and the raw path was valid; the common LOS/smoothing stage introduced a small local clearance shortfall under the fixed 0.025 m safety allowance.

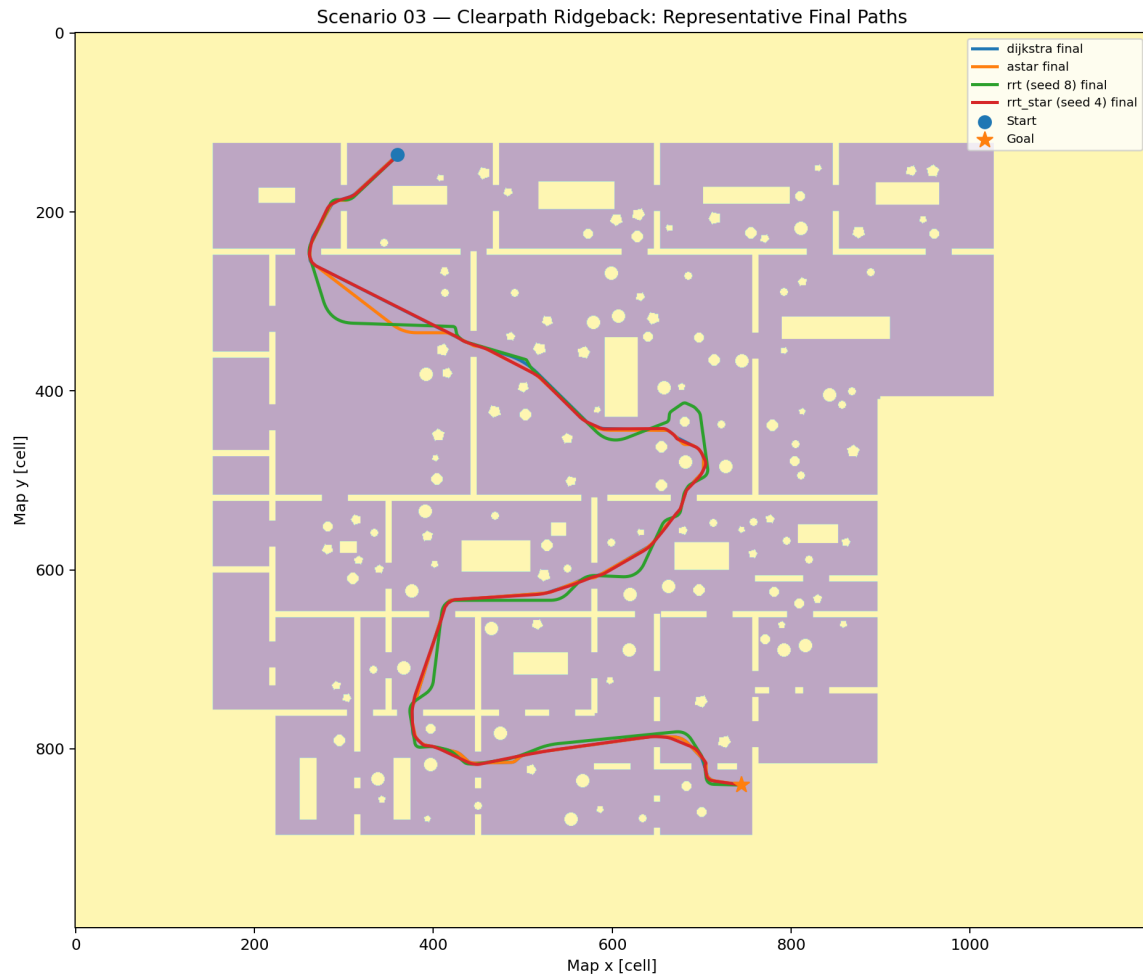


Figure 25: Representative Ridgeback final paths. The four planners use broadly similar global corridor structure, while local geometry differs around bottlenecks and temporary-object clusters.

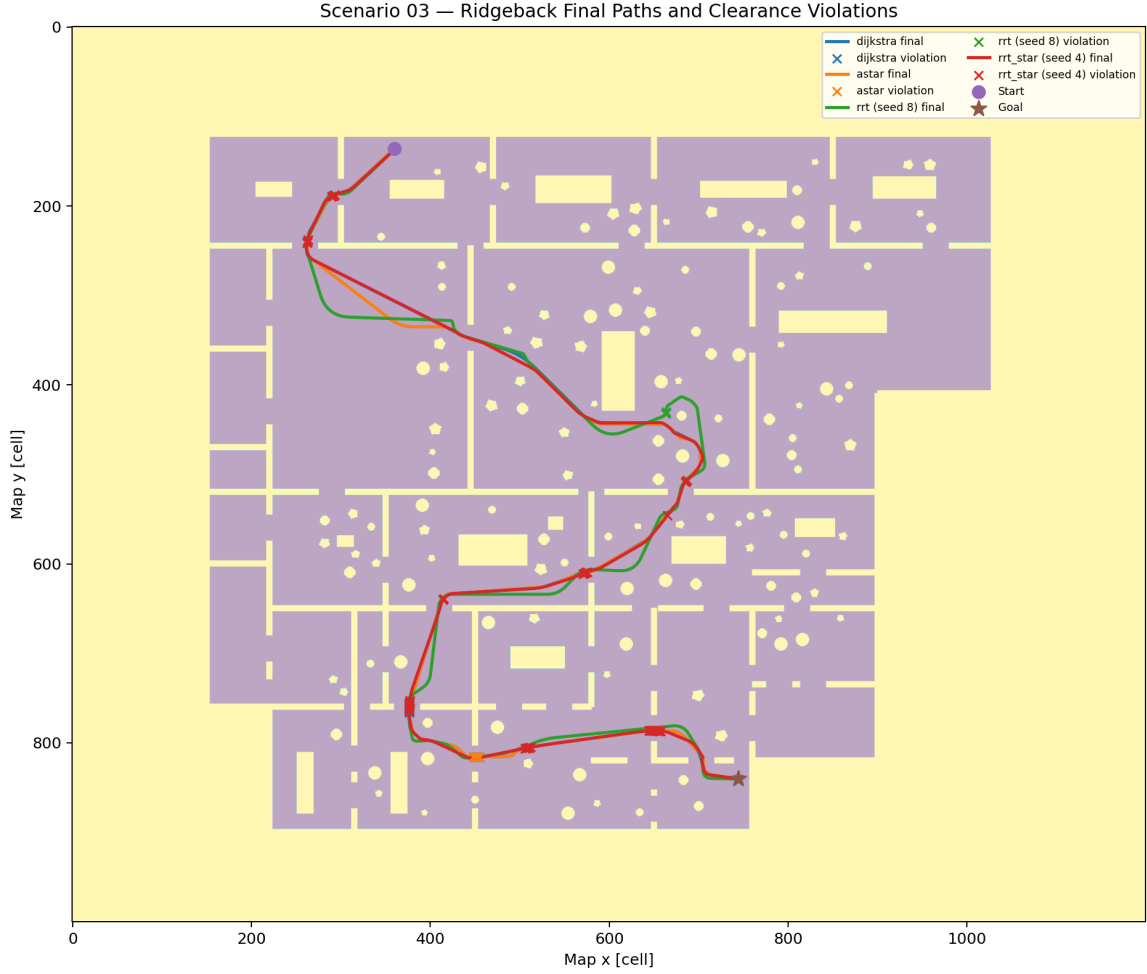


Figure 26: Ridgeback final trajectories with continuous-clearance violation samples marked by crosses. The crosses are robot-center locations, not obstacle pixels; a violation occurs when the centerline enters the robot’s inflated forbidden region.

19.2 RRT and RRT* search difficulty

The larger Ridgeback footprint qualitatively changes the stochastic planning problem. RRT and RRT* each succeeded in only 7 of 10 seeds within the 50,000-iteration budget. For successful trials, the median first-feasible iteration was 32,140, approximately 6.27 times the Waffle RRT median.

No Ridgeback RRT* seed produced a solution at 5,000, 10,000, or 20,000 iterations. At 50,000 iterations, only seven seeds had a solution. This result is important because the planner code, start/goal pair, stochastic seeds, and high-level map are unchanged. The difference arises from robot-specific free-space erosion: footprint changes not only path clearance but search difficulty and reliability.

19.3 RRT mission quality

For the seven successful Ridgeback RRT trials, median final path length was 86.69 m and median safety-aware mission time was 222.10 s. None of the successful raw or final RRT trajectories passed the continuous-clearance criterion. The median maximum final violation was approximately 22.6 mm and the largest observed value was approximately 29.1 mm.

RRT therefore retains its expected strength—eventual first-feasible discovery in many seeds—but its reliability, route quality, and continuous-clearance robustness are poor for this larger-footprint problem.

19.4 RRT* mission quality

Successful Ridgeback RRT* trials produced a median final path length of 79.375 m and median mission time of 203.552 s, close to the deterministic routes. However, median planning time was 169.80 s, approximately 57 times A* planning time. None of the seven successful raw or final RRT* trajectories passed the stricter continuous-clearance criterion. Median maximum final violation was approximately 24.9 mm, and the largest observed value was 43.8 mm.

Relative to A*, median RRT* final path length improved by only about 0.439 m and median mission time by about 1.77 s. The result again shows that lower planner cost does not automatically imply stronger engineering value, particularly when reliability, clearance, and computation are considered together.

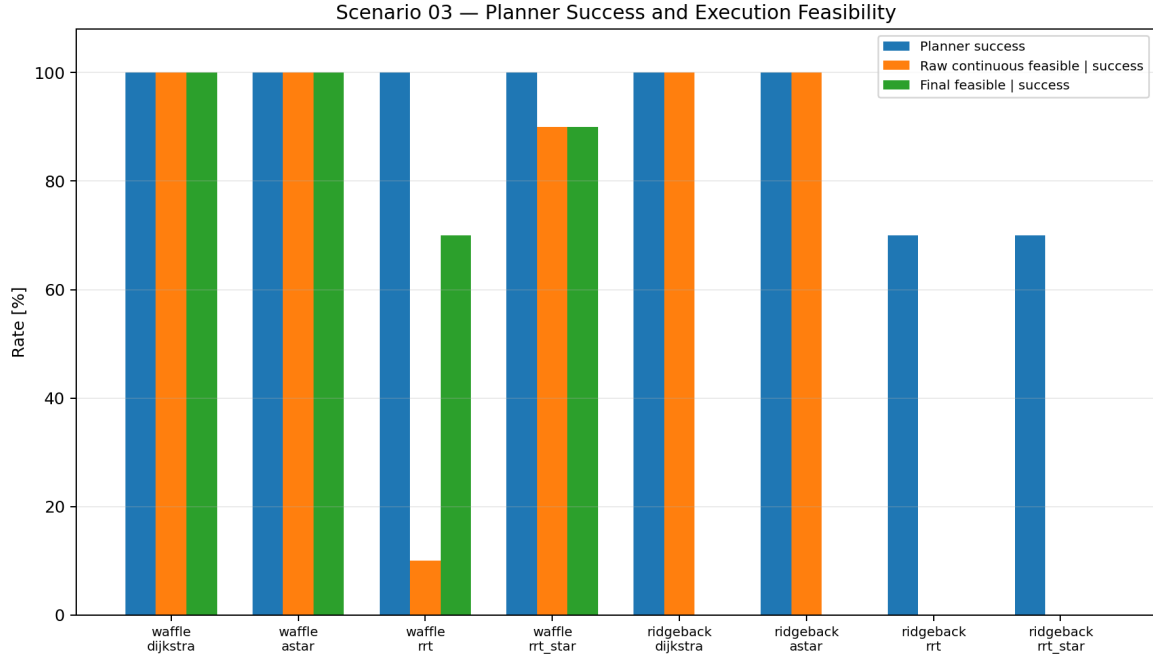


Figure 27: Scenario 3 separation between planner success, raw continuous feasibility, and final trajectory feasibility. The distinction is especially important for the Ridgeback sampling-based results.

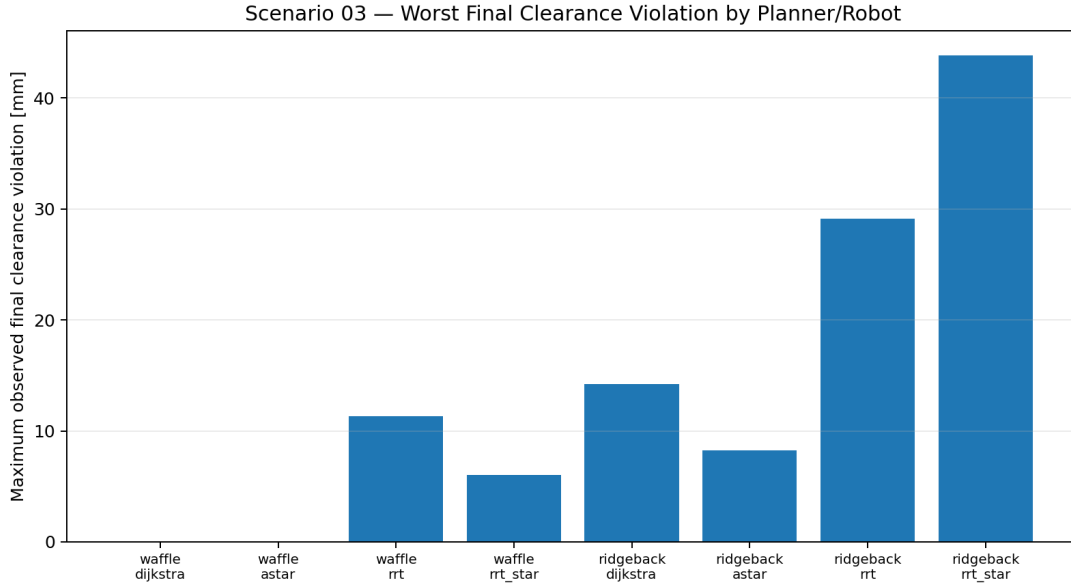


Figure 28: Worst observed final clearance violation by planner and robot. A zero value indicates that no successful final trajectory violated the common continuous-clearance criterion.

20 Robot Footprint Changes Planner Search Difficulty

The Scenario 3 stochastic results show that footprint affects more than geometric clearance. It also changes the difficulty of discovering a route. With the Waffle configuration space, both RRT and RRT* found a solution in all ten seeds, with a median first-feasible iteration of approximately 5,126. With the Ridgeback configuration space, the same planner parameters and 50,000-iteration budget produced solutions in only seven of ten seeds, and the median first-feasible iteration among successful trials increased to approximately 32,140.

This is a qualitative change in the search problem rather than a simple path-length increase. The larger footprint erodes narrow free-space connections and reduces the volume of useful samples and collision-free extensions. Consequently, the same physical map, mission, and stochastic planner can exhibit substantially different reliability solely because the robot-specific configuration space has changed.

The result extends the Scenario 1 observation. Scenario 1 showed that robot footprint can change *route access*; Scenario 3 shows that robot footprint can also change *planner reliability and convergence behavior*.

21 Planning Computation Versus Mission Benefit

A central reason for carrying planner output through to mission time is that planner effort and execution benefit occur on different time scales. The planner may spend hundreds of seconds improving a path whose execution time changes by only a few seconds.

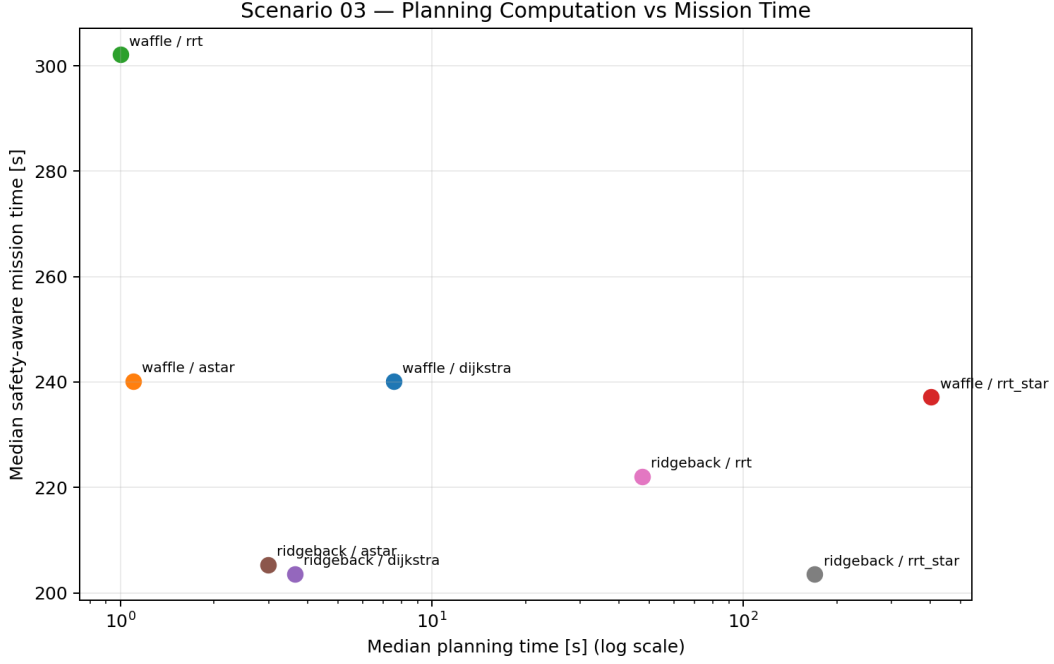


Figure 29: Median planning computation versus median safety-aware mission time. The horizontal axis is logarithmic. RRT* reaches strong geometric path quality but at substantially higher computational cost than A* in both robot cases.

For Waffle, median RRT* planning is approximately 365 times A* planning for a median mission-time reduction of about 3.0 s. For Ridgeback, median RRT* planning is approximately 57 times A* planning for a median mission-time reduction of about 1.77 s, while planner success falls to 70% and continuous feasibility remains poor.

The relevant engineering question is therefore not whether more computation can improve geometric path cost—it can—but whether the improvement is large enough and reliable enough to justify that computation for the intended task.

22 Integrated Engineering Findings

The three scenarios form a single chain of evidence rather than three independent demonstrations.

22.1 Finding 1: robot geometry changes feasibility before planning

Scenario 1 shows that a route may be available to a compact robot yet physically unavailable to a larger robot. Rectangular-footprint verification prevents this result from being dismissed as a circular-inflation artifact.

22.2 Finding 2: ordinary clutter can amplify robot-dependent route cost

Scenario 2 demonstrates that temporary occupancy need not make a mission impossible to matter operationally. Waffle path length remains nearly unchanged, while Ridgeback path length and direction changes increase sharply. Environmental clutter changes how much of a platform’s nominal performance is usable.

22.3 Finding 3: planner success is not execution feasibility

Scenario 3 separates native planner collision checks from common continuous-clearance validation and from post-processing feasibility. This distinction exposes small local margin failures that would be hidden by a single planner-success flag.

22.4 Finding 4: path-length optimization is not safety optimization

RRT* optimizes geometric path cost under its own collision model. It does not optimize continuous safety margin. Ridgeback RRT* can therefore produce path lengths competitive with A* while still exhibiting larger local clearance violations.

22.5 Finding 5: footprint changes search difficulty itself

Waffle sampling planners succeed in all ten final seeds, while Ridgeback RRT and RRT* succeed in only seven. Ridgeback first-feasible iteration count is also much larger. Configuration-space erosion therefore changes not only the final route but the stochastic search problem.

22.6 Finding 6: the best geometric path need not be the best engineering choice

Under the tested benchmark assumptions, A* provides the strongest overall trade. It is deterministic, computationally inexpensive, produces raw paths that satisfy the continuous criterion for both robots, and achieves mission times close to or better than much more expensive alternatives. RRT* can produce slightly shorter paths, particularly for Waffle, but its additional computation yields comparatively small mission-level gains.

This conclusion is intentionally limited to the tested benchmark. It is not a universal claim that A* is superior to RRT* in all robotic planning problems. Different state dimensions, nonholonomic constraints, continuous obstacle geometry, kinodynamic planning requirements, or highly irregular search spaces may change the trade completely.

23 Limitations and Interpretation Boundaries

- **Orientation-independent global footprint.** Global planning uses a conservative circumscribed-circle representation. The benchmark does not perform full SE(2) polygon-footprint search.
- **Temporary objects are static snapshots.** They represent occupancy that may arise during operation, but no object trajectory prediction, reciprocal avoidance, or local dynamic replanning is modeled.
- **No local planner or closed-loop tracking.** The study evaluates global paths, common refinement, clearance, and kinematic mission-time estimates. Controller tracking error, localization error, and reactive obstacle avoidance are outside the present scope.
- **Ridgeback motion model is platform-level.** The benchmark uses a simplified translational/normal-acceleration approximation rather than wheel-level mecanum dynamics, tire forces, or actuator saturation.
- **Safety policy is a benchmark assumption.** The 0.025 m geometric allowance, 1.0 m caution band, 80% open-space cruise, and 0.30 m/s caution cap are explicit experimental policies, not claims of universal industrial safety requirements.
- **Sampling-planner implementation is intentionally straightforward.** Nearest and near-neighbor searches use linear scans rather than an optimized spatial index. Reported planning times therefore describe the implemented system, not the best theoretically achievable RRT/RRT* runtime.
- **Ten stochastic seeds are an engineering sample, not a population-level statistical study.** They are sufficient to expose seed sensitivity and reliability differences in a portfolio-scale benchmark, but they do not constitute exhaustive statistical characterization.
- **Continuous clearance still depends on map resolution and field construction.** The benchmark is stricter than planner-native occupancy checking, but it is not equivalent to hardware collision certification.

24 Reproducibility and Data Products

The final benchmark stores raw trial data separately from summary statistics. The principal Scenario 3 outputs included with this report package are

```
data/scenario03_planner_trials.csv
data/scenario03_planner_summary.csv
data/scenario03_rrt_star_convergence.csv
data/scenario03_final_benchmark.log
data/scenario03_visualization_summary.txt
```

Sampling-based planners use explicit random seeds. The final stochastic benchmark is therefore defined by the combination

Map + Mission + Robot C-Space + Planner Parameters + Seed.

Representative stochastic paths used in the visualization were not selected for appearance. Among successful trials, the seed whose safety-aware mission time was closest to the successful-trial median was used. This produced Waffle RRT seed 0, Waffle RRT* seed 0, Ridgeback RRT seed 8, and Ridgeback RRT* seed 4.

25 Conclusion

The project began with a conventional planner-validation problem and ended with a broader engineering question about whether a returned path is actually meaningful for a physical robot.

Algorithm-level validation established trustworthy Dijkstra, A*, RRT, and corrected RRT* implementations. Moving to a metric environment then showed that robot footprint changes configuration space before any planner is selected. Scenario 1 demonstrated route divergence caused by footprint alone. Scenario 2 showed that temporary clutter can amplify the effect dramatically, especially for the larger Ridgeback. Scenario 3 completed the study by subjecting all four planners to the same safety-aware path-to-execution pipeline.

The final benchmark separates four questions that are often collapsed into one:

Can the robot physically perform the mission?
Can the planner discover a route reliably?
Does the returned path remain feasible after refinement?
Is the computation justified by mission-level benefit?

The strongest general result is that **planning quality is not path length alone**. A useful navigation solution must be interpreted through reliability, physical clearance, post-processing behavior, computation, and execution cost. Under the present assumptions, A* provides the most balanced engineering result. RRT remains useful as a rapid stochastic feasibility mechanism but exhibits high path variability. RRT* demonstrates excellent geometric optimization for Waffle, yet its hundreds of seconds of planning buy only a few seconds of mission improvement, while the larger Ridgeback exposes reliability and clearance limitations.

The study therefore supports a system-level view of path planning:

Engineering Planning Quality = $f(\text{Reliability, Feasibility, Computation, Execution Cost})$

This is the decision framework that the benchmark was designed to produce.

Appendices

A Software Module Reference

The final repository retains the modular separation developed throughout the study. Table 9 lists the principal modules and their engineering responsibilities, including the Scenario 3 components added after the original Part 1 report.

Table 9: Expanded software module reference.

Module	Responsibility
<code>benchmark/map_loader.py</code>	Parses Moving AI benchmark maps into the common binary occupancy representation.
<code>benchmark/scenario_loader.py</code>	Loads benchmark start/goal pairs and externally supplied reference lengths.
<code>benchmark/metric_map_loader.py</code>	Loads ROS/Nav2-style metric maps while preserving resolution, origin, thresholds, and unknown-space treatment.
<code>benchmark/runner.py</code>	Original algorithm-level repeated-run benchmark harness.
<code>benchmark/mission_benchmark.py</code>	Generates reproducible common start/goal missions for cross-platform feasibility studies.
<code>planners/base_planner.py</code>	Defines the shared 8-connected grid motion model, costs, bounds, and no-corner-cutting rule.
<code>planners/geometry.py</code>	Provides continuous point/edge collision checking and Euclidean path utilities for sampling-based planners.
<code>planners/dijkstra.py</code>	Deterministic reference graph search.
<code>planners/astar.py</code>	Octile-heuristic graph search using the same legal edge set as Dijkstra.
<code>planners/rrt.py</code>	Goal-biased stochastic tree growth with complete-edge collision checking and first-feasible termination.
<code>planners/rrt_star.py</code>	RRT* parent selection, rewiring, corrected descendant-cost propagation, goal-candidate re-evaluation, and convergence logging.
<code>robots/robot_model.py</code>	Loads robot geometry and motion limits and computes derived footprint quantities.
<code>robots/footprint.py</code>	Converts physical robot dimensions into conservative configuration-space inflation.
<code>robots/feasibility.py</code>	Robot-specific geometric/kinematic feasibility calculations.
<code>metrics/connectivity_metrics.py</code>	Connected-component and free-space structure analysis.
<code>metrics/mission_feasibility.py</code>	Distinguishes invalid start/goal configurations from true connectivity failures.
<code>benchmark/scenario03_safety_inflation.py</code>	Screens common safety allowances against robot-specific start/goal validity and route existence.
<code>benchmark/scenario03_temporary_objects.py</code>	Extracts the temporary/caution-object mask and distance field by differencing dense and structural maps.
<code>benchmark/scenario03_trajectory_validation.py</code>	Development diagnostic separating raw-path feasibility from LOS/smoothing effects.

Module	Responsibility
<code>benchmark/scenario03_proximity_speed.py</code>	Implements the open-space and caution-zone speed policy and associated diagnostics.
<code>benchmark/scenario03_common_evaluator.py</code>	Planner-independent LOS, smoothing, resampling, continuous-clearance validation, violation metrics, motion limits, caution policy, and mission-time evaluation.
<code>benchmark/scenario03_rrt_pilot.py</code>	High-budget stochastic pilot used to identify useful iteration ranges and expose high-budget computational cost.
<code>benchmark/scenario03_planner_benchmark.py</code>	Final Dijkstra/A*/RRT/RRT* benchmark with ten stochastic seeds, checkpoint saving, convergence export, and summary generation.
<code>benchmark/scenario03_path_visualization.py</code>	Selects median-representative stochastic seeds and recreates planner overlays, RRT variability, RRT* convergence paths, and violation figures.

B Development Diagnostics and Traceability

B.1 Safety-margin screening

Before fixing the Scenario 3 allowance at 0.025 m, several candidate margins were screened. The purpose was not to optimize a planner score but to identify whether the common mission remained geometrically valid for both robot platforms. Ridgeback became start/goal-invalid by 0.05 m of additional inflation, whereas the selected 0.025 m value preserved the mission and remained directly tied to the 0.05 m map resolution.

B.2 RRT/RRT* high-budget pilot

A three-seed, 50,000-iteration pilot was run before the final ten-seed benchmark. Waffle RRT first solutions appeared at approximately 4,045–5,961 iterations; Ridgeback successful trials did not find their first solution until approximately 30,867–32,140 iterations, and one of three Ridgeback seeds still failed at 50,000 iterations. RRT* required approximately 400 s per 50,000-iteration Waffle trial in the straightforward linear-neighbor implementation. This inefficiency was intentionally measured rather than hidden: it provided the evidence needed to interpret diminishing returns in the final study.

B.3 Validation-method diagnostic

During development, an earlier nearest-cell dense validation and the final bilinear clearance-field validation did not classify every marginal sample identically. The final benchmark therefore uses one common method consistently: bilinear sampling of the continuous clearance field after uniform trajectory resampling. The earlier diagnostic figure is retained only to document the evolution of the validation method and must not be mixed numerically with the final benchmark results.

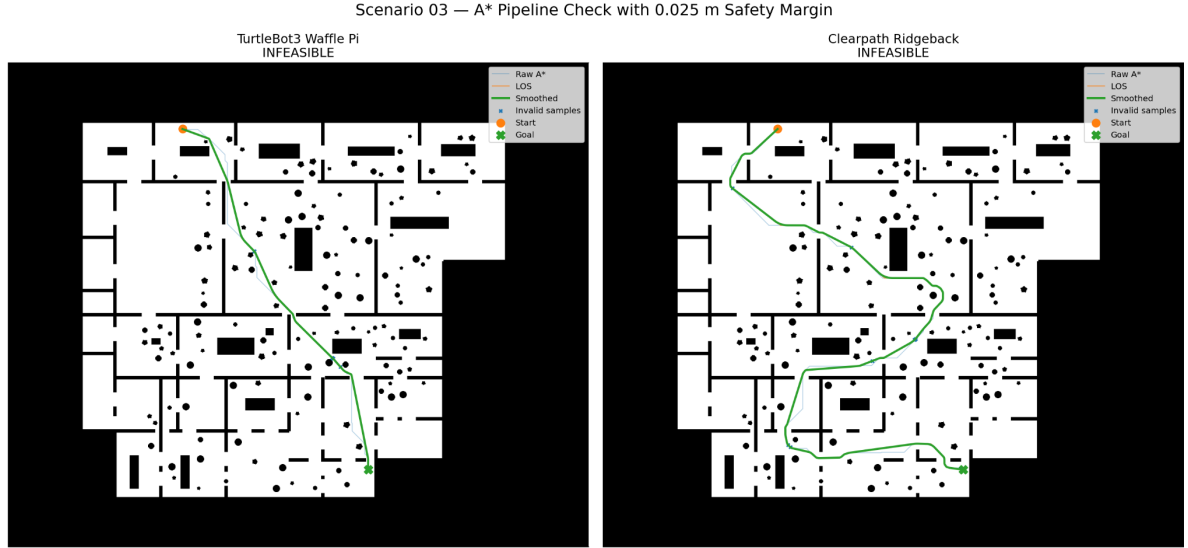


Figure 30: Development-only A* trajectory validation diagnostic. This figure uses an earlier clearance-sampling implementation and is retained for traceability. Final Scenario 3 classifications use the common bilinear clearance evaluator described in the main text.

B.4 Preliminary Scenario 1 timing model

An earlier rest-to-rest segment timing model was used during Scenario 1 debugging before the final continuous velocity parameterization was introduced. It produced approximately 200.2 s for Waffle and 67.1 s for Ridgeback. Because that formulation forced rest-to-rest behavior at LOS segment boundaries, it is retained only as development traceability and is not used in final conclusions.

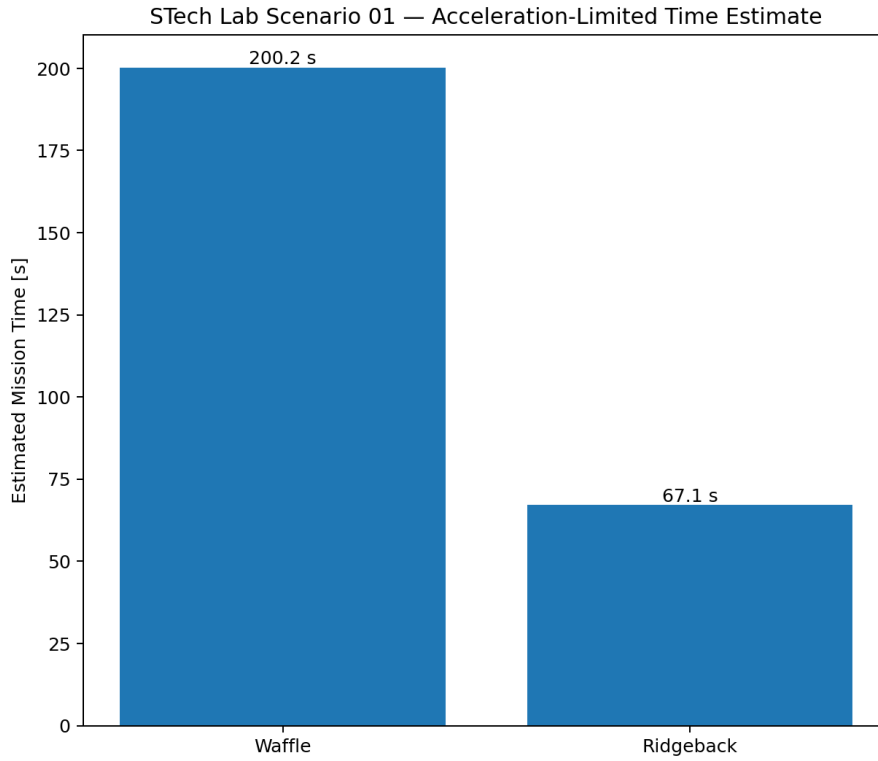


Figure 31: Preliminary Scenario 1 rest-to-rest timing model retained for development traceability. Final conclusions use the continuous velocity profiles and the later Scenario 3 operating policy.

C Full Scenario 3 Trial Tables

The following tables preserve every final benchmark trial. Deterministic planners have no seed or iteration budget entry. For sampling-based planners, planner success is reported separately from raw continuous feasibility and final post-processed feasibility.

Table 10: Full Scenario 3 trials: TurtleBot3 Waffle Pi.

Planner	Seed	Succ.	Plan [s]	Iter.	Raw [m]	Final [m]	Raw ok	Final ok	Viol. [mm]	Mission [s]
dijkstra	—	Y	7.55	—	44.47	42.13	Y	Y	0.0	240.08
astar	—	Y	1.10	—	44.47	42.14	Y	Y	0.0	240.16
rrt	0	Y	1.14	5961	64.58	52.76	N	N	8.7	300.11
rrt_star	0	Y	399.62	50000	41.72	41.67	Y	Y	0.0	237.17
rrt	1	Y	0.36	4045	66.99	55.21	N	Y	0.0	313.99
rrt_star	1	Y	399.02	50000	41.74	41.67	Y	Y	0.0	237.21
rrt	2	Y	0.80	4569	73.10	57.24	N	N	11.3	326.00
rrt_star	2	Y	409.76	50000	41.66	41.63	Y	Y	0.0	236.98
rrt	3	Y	0.87	4695	58.83	48.74	N	Y	0.0	277.61
rrt_star	3	Y	410.40	50000	41.71	41.64	Y	Y	0.0	237.26
rrt	4	Y	2.33	7831	65.64	51.89	N	N	8.1	295.30
rrt_star	4	Y	388.10	50000	41.73	41.64	Y	Y	0.0	237.15
rrt	5	Y	0.44	3830	60.93	51.57	N	Y	0.0	293.41
rrt_star	5	Y	417.68	50000	41.72	41.66	N	N	6.0	237.17
rrt	6	Y	0.37	3809	65.04	54.76	N	Y	0.0	311.47
rrt_star	6	Y	406.86	50000	41.76	41.70	Y	Y	0.0	237.40
rrt	7	Y	5.71	13239	66.58	53.44	N	Y	0.0	304.19
rrt_star	7	Y	349.00	50000	41.71	41.66	Y	Y	0.0	237.07
rrt	8	Y	1.13	5841	73.90	55.12	Y	Y	0.0	313.54
rrt_star	8	Y	402.43	50000	41.69	41.65	Y	Y	0.0	237.13
rrt	9	Y	1.15	5557	52.16	45.42	N	Y	0.0	258.49
rrt_star	9	Y	401.53	50000	41.68	41.63	Y	Y	0.0	237.05

Table 11: Full Scenario 3 trials: Clearpath Ridgeback.

Planner	Seed	Succ.	Plan [s]	Iter.	Raw [m]	Final [m]	Raw ok	Final ok	Viol. [mm]	Mission [s]
dijkstra	—	Y	3.63	—	83.66	79.41	Y	N	14.2	203.59
astar	—	Y	2.98	—	83.66	79.81	Y	N	8.2	205.33
rrt	0	N	98.69	50000	—	—	N	N	—	—
rrt_star	0	N	171.90	50000	—	—	N	N	—	—
rrt	1	Y	26.96	30867	99.71	84.92	N	N	22.6	211.07
rrt_star	1	Y	167.35	50000	79.50	79.26	N	N	22.6	202.81
rrt	2	Y	31.38	32140	104.27	86.69	N	N	22.6	221.26
rrt_star	2	Y	189.43	50000	81.67	81.40	N	N	34.4	210.71
rrt	3	Y	26.35	29176	99.83	83.39	N	N	22.6	213.09
rrt_star	3	Y	205.57	50000	79.47	79.18	N	N	43.8	203.17
rrt	4	Y	46.72	34379	122.84	101.81	N	N	29.1	282.74
rrt_star	4	Y	220.99	50000	79.61	79.37	N	N	24.9	203.55
rrt	5	N	92.01	50000	—	—	N	N	—	—
rrt_star	5	N	160.16	50000	—	—	N	N	—	—
rrt	6	Y	48.12	46755	105.79	88.10	N	N	28.4	227.59
rrt_star	6	Y	101.86	50000	79.80	79.41	N	N	23.3	204.93
rrt	7	Y	18.55	22735	111.39	88.67	N	N	22.6	226.78
rrt_star	7	Y	209.39	50000	79.41	79.19	N	N	37.9	204.59
rrt	8	Y	76.00	45614	107.31	86.64	N	N	22.6	222.10
rrt_star	8	Y	167.70	50000	79.73	79.40	N	N	22.6	202.09
rrt	9	N	68.06	50000	—	—	N	N	—	—
rrt_star	9	N	116.40	50000	—	—	N	N	—	—

D RRT* Convergence Tables

The convergence export stores the best raw goal cost available at each iteration checkpoint. These are planner-cost checkpoints; they are not mission-time snapshots. Mission-time convergence for the representative Waffle seed was obtained by explicit reruns at the selected budgets and is reported in the main text.

Table 12: Waffle RRT* best raw goal cost [m] at each iteration checkpoint.

Seed	5k	10k	20k	50k
0	—	46.27	41.96	41.72
1	51.31	44.94	41.95	41.74
2	54.19	44.25	41.96	41.66
3	46.28	43.95	42.26	41.71
4	—	47.95	43.37	41.73
5	46.01	44.08	43.54	41.72
6	50.77	42.72	41.96	41.76
7	—	—	44.01	41.71
8	—	50.56	41.97	41.69
9	—	44.25	41.81	41.68

Table 13: Ridgeback RRT* best raw goal cost [m] at each iteration checkpoint.

Seed	5k	10k	20k	50k
0	—	—	—	—
1	—	—	—	79.50
2	—	—	—	81.67
3	—	—	—	79.47
4	—	—	—	79.61
5	—	—	—	—
6	—	—	—	79.80
7	—	—	—	79.41
8	—	—	—	79.73
9	—	—	—	—

E Reproducibility Commands and Data Products

The final benchmark stores raw trials, aggregate statistics, convergence history, and console logs separately so that figures and conclusions can be traced back to numerical results.

E.1 Primary data products

```
data/scenario03_planner_trials.csv
data/scenario03_planner_summary.csv
data/scenario03_rrt_star_convergence.csv
data/scenario03_final_benchmark.log
data/scenario03_visualization.log
data/scenario03_visualization_summary.txt
```

E.2 Representative execution commands

From the repository root, the final benchmark and visualization were executed as follows:

```

mkdir -p results/stech_lab/scenario03/final_benchmark

python3 -u -m benchmark.scenario03_planner_benchmark \\\
  2>&1 | tee results/stech_lab/scenario03/final_benchmark/\\
scenario03_final_benchmark.log

python3 -u -m benchmark.scenario03_path_visualization \\\
  2>&1 | tee results/stech_lab/scenario03/final_benchmark/visualization/\\
scenario03_visualization.log

```

E.3 Random-seed definition

Sampling experiments are reproducible under the combination

Map + Mission + Robot C-Space + Planner Parameters + Seed.

The final stochastic benchmark used seeds 0–9 for both RRT and RRT*. Representative visualization seeds were selected by choosing, among successful trials, the mission time closest to the successful-trial median: Waffle RRT seed 0, Waffle RRT* seed 0, Ridgeback RRT seed 8, and Ridgeback RRT* seed 4. This rule avoids visual cherry-picking.

E.4 Scenario summary

Table 14: Final scenario progression retained for report traceability.

Scenario	Name	Engineering purpose
1	Footprint-driven route divergence	Static metric environment and fixed mission; isolates geometry-driven route availability.
2	Dense temporary clutter	Adds 132 temporary/caution objects; tests how clutter amplifies robot-dependent route cost and maneuvering.
3	Safety-aware planner benchmark	Adds common safety allowance, continuous validation, operating-speed policy, and multi-planner comparison.